

Velocity Corporate Training Center, Pune

Generative AI - Retrieval-Augmented Generation (RAG)



Content

1. Introduction to RAG

- What is Retrieval-Augmented Generation (RAG)?
 - Why do we need RAG?
 - High-level architecture diagram
-

2. LLM Limitations and Enhancement Strategies

- Challenges with LLMs in Isolation
 - Enhancing LLMs via Fine-Tuning
 - Augmenting LLMs with Retrieval (RAG)
 - When to Use: LLM-only vs Fine-Tuning vs RAG
-

3. RAG Architecture

- End-to-End Pipeline Flow
 - Core Components (Retriever + Generator)
 - Prompt Construction Pipeline
-

4. Document Ingestion Pipeline

- Preprocessing
 - Handling PDFs (scanned/readable), DOCX, audio
 - Chunking Strategies
 - Fixed-size, overlapping, semantic, metadata-aware
 - Embedding and Storage
 - Embedding models (e.g., text-embedding-ada-002)
 - Pooling strategy and dimensionality (1536D)
 - Metadata Strategy and Schema Design
-

5. Retrieval Strategies

- Dense, Sparse, and Hybrid Retrieval
 - Top-K vs MMR vs Multi-query vs Self-query
 - Contextual Compression Retriever
 - Pune Tourism Use Case (Chunk-to-Retriever Mapping)
-

6. Prompt Engineering in RAG

- Prompt Types: System, User, Assistant
- Prompt Construction Best Practices
- Output Formatting
 - JSON, bullet points, tables
- Zero-shot and Few-shot Prompting
- Chain-of-Thought (CoT) Prompting
- Role Prompting
- Multi-turn Prompting

- Hallucination Reduction Techniques
-

7. Evaluation and Testing

- Importance of Evaluation in RAG
 - Retriever Evaluation
 - Metrics: Recall@K, Precision@K, Coverage, Diversity
 - Generator Evaluation
 - Metrics: Faithfulness, Factual Accuracy, Completeness, Clarity
 - End-to-End Evaluation Strategy
 - Evaluation Tools
 - Manual QA, LLM-as-a-Judge, RAGAS, TruLens, OpenAI Evals
-

8. Tools and Frameworks

- Preprocessing: spaCy, PyMuPDF, Tika, Azure Form Recognizer, AWS Textract
 - Embedding Models: OpenAI, Azure OpenAI, Cohere, HuggingFace, Amazon Titan
 - Vector Databases:
 - Open-source: FAISS, Chroma, Milvus, Weaviate
 - Managed: Pinecone, Azure AI Search, Amazon OpenSearch, Amazon Kendra, Redis Vector
 - Orchestration: LangChain, LlamaIndex, Haystack, Semantic Kernel, Bedrock Agents
 - Evaluation: RAGAS, TruLens, LangChain Eval, OpenAI Evals, Azure Monitor, AWS CloudWatch
-

9. Security, Privacy, and Compliance

- Key Risks: PII Leakage, Unauthorized Access, Prompt Injection, Data Residency
 - PII Examples and Redaction Strategy
 - Access Control using Metadata Filtering
 - Authentication and Authorization (Azure AD, Cognito, OAuth2)
 - Prompt Guardrails and Moderation Tools
 - Encryption (Azure Key Vault, AWS KMS)
 - Audit Logging and Traceability
 - Compliance Frameworks: GDPR, HIPAA, SOC 2, ISO 27001, FINRA
-

10. Best Practices

- Chunking & Preprocessing
 - Embedding Normalization and Metadata Strategy
 - Retrieval Optimization (Hybrid, Self-query, MMR)
 - Prompt Structure and Compression
 - Evaluation Loop and Continuous Testing
 - Token Management and Cost Control
 - Logging, Versioning, and Feedback Loops
-

11. Common Challenges

- Irrelevant or Redundant Chunks
 - Retrieval Drift / Context Misalignment
 - Prompt Overflow and Token Limit Issues
 - Latency and Real-time Performance
 - Hallucination Despite Accurate Context
 - Edge-case Failures (query phrasing, special tokens)
 - Multi-turn Follow-up Confusion
 - Document Update & Re-indexing Complexity
 - Cost Management (Token + Infra)
-

12. Real-World Use Cases

- HR Policy Assistant
- Customer Service Chatbot (Insurance)
- Legal Document QA Assistant
- Clinical Decision Support System
- University Knowledge Assistant
- Internal Finance Policy Assistant
- Research Summarization Assistant
- Compliance Audit Assistant
- E-commerce Product Support Bot
- Enterprise Software Documentation Assistant

1. Introduction to RAG

◆ What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is an architecture that combines two core capabilities:

- **Retrieval:** Fetches relevant external documents or knowledge snippets based on a user's query.
- **Generation:** Uses a Large Language Model (LLM) to generate a response grounded in the retrieved content.

Instead of relying solely on the LLM's internal training data, RAG enhances the model's performance by giving it access to **real-time, external knowledge sources** like databases, PDFs, web pages, or internal documents.

Example: When asked, "What is the university's retirement policy?", a RAG system first retrieves the relevant HR policy document and then generates a factual, grounded answer based on it.

RAG systems typically involve:

- **Embeddings:** Converting text to vectors for semantic search
- **Vector Databases:** Storing and searching document chunks
- **Prompt Construction:** Augmenting the user query with retrieved chunks
- **LLM Generation:** Producing a natural language response using the augmented prompt

◆ Why Do We Need RAG?

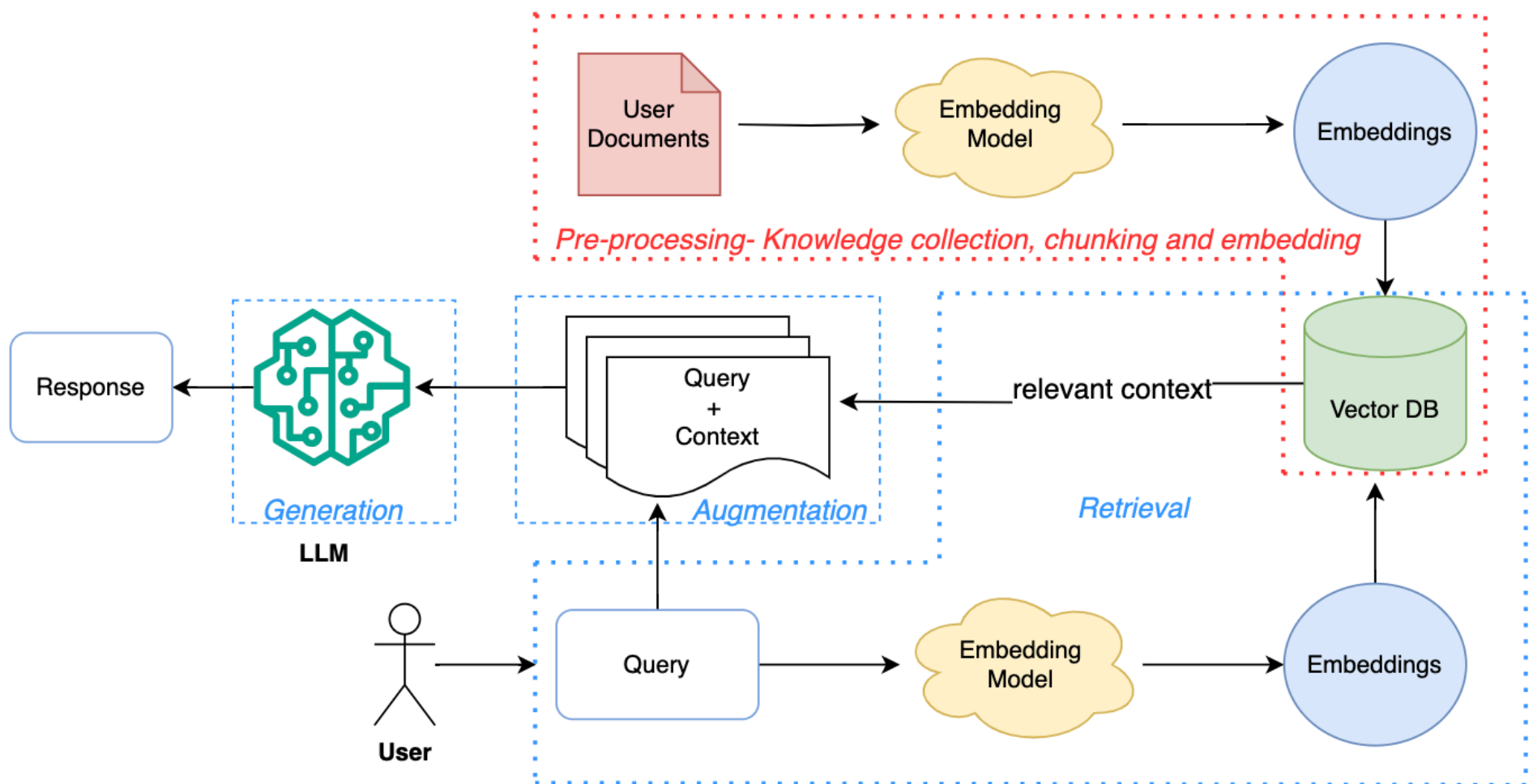
While modern LLMs are powerful, they have critical limitations when used alone. RAG addresses these by enabling the model to **retrieve real, relevant data dynamically**, rather than depending on outdated or memorized content.

✓ Key Reasons We Need RAG:

- **Real-Time Knowledge Access:** LLMs have a cutoff date for training; RAG can fetch up-to-date content anytime.
- **Reduced Hallucination:** By grounding the output in retrieved content, RAG helps reduce factually incorrect answers.
- **Domain Adaptability:** No need to fine-tune; just add new documents to the vector store to handle new topics.
- **Explainability:** Users can trace the model's answer back to specific source documents.
- **Lower Maintenance Cost:** Instead of retraining a model, you just update your documents.

💡 RAG shifts the paradigm from "What the model remembers" to "What the model can look up."

◆ High-level architecture diagram of RAG ?



2. LLM Limitations and Enhancement Strategies

◆ Challenges with LLMs in Isolation

Large Language Models (LLMs) like GPT-4 are powerful, but when used without any augmentation, they face critical limitations:

- **Static Knowledge:** LLMs are trained on fixed datasets. They cannot access real-time information after their training cutoff.
- **Hallucination:** LLMs can produce fluent but factually incorrect responses, especially when asked about niche, ambiguous, or proprietary topics.
- **No Source Grounding:** Responses generated by LLMs can't be traced back to a source, which makes auditing, validation, and trust difficult.
- **Limited Domain Adaptation:** Out-of-the-box LLMs may perform poorly when asked to answer questions based on internal business documents or industry-specific terminology.
- **Costly to Update:** Updating an LLM's knowledge or behavior often requires full fine-tuning or re-training, which is resource-intensive.
- **Prompt Limitations:** LLMs can only process a limited number of tokens in a prompt, restricting how much background context can be included.
- **Compliance Risks:** In regulated industries, the inability to cite sources or show the origin of an answer makes LLMs risky to deploy without augmentation.

◆ Enhancing LLMs via Fine-Tuning

Fine-tuning is a method of adapting a pre-trained LLM to a specific use case by training it further on a domain-specific or task-specific dataset.

✓ When to use fine-tuning:

- To align the model's tone, style, or behavior to match your organization
- When your use case involves highly structured, repetitive outputs
- To improve performance on specific tasks like classification, summarization, or entity extraction

⚖️ Pros:

- Personalizes model behavior
- Can improve performance on narrowly defined tasks

✗ Cons:

- Expensive (requires GPUs and ML engineering)
- Time-consuming to train and deploy
- Still doesn't solve real-time knowledge access or source traceability
- Updating requires another fine-tuning cycle

◆ Augmenting LLMs with Retrieval (RAG Approach)

Retrieval-Augmented Generation (RAG) enhances LLMs by giving them **dynamic access to external documents**. Instead of relying on memory (like traditional LLMs), RAG enables the model to **search and retrieve relevant information at query time**.

 How it works:

- The user’s query is embedded and used to search a vector database
- The top relevant document chunks are retrieved
- These chunks are added to the prompt
- The LLM generates a response grounded in this context

 Benefits of RAG:

- No need to fine-tune the model for every knowledge update
- Improves accuracy by grounding responses in retrieved content
- Enables domain-specific answers using your document base
- Easier to audit and explain responses

◆ **When to Use: LLM-only vs Fine-Tuning vs RAG**

Scenario	LLM-only	Fine-Tuning	RAG
General-purpose conversation	✓	✗	✗
Repetitive structured task (e.g., resumes)	✗	✓	✗
Domain-specific document Q&A	✗	✗	✓
Need real-time document updates	✗	✗	✓
Must trace answers to sources (audit/compliance)	✗	✗	✓
Need to adjust tone or personality	✗	✓	✗

 In many enterprise settings, **RAG is the preferred solution** when knowledge needs to stay current, explainable, and cost-effective without retraining.

3. RAG Architecture

◆ **End-to-End Flow Overview**

RAG connects a retrieval system with a language model to answer user queries using external documents.

Simplified Pipeline:

Query → Embed → Retrieve → Augment → Generate → Respond

Each stage transforms the input progressively, ensuring the final output is accurate and context-aware.

◆ **Core Components**

- **Embedder:** Converts queries and documents into semantic vectors.
- **Vector Database:** Stores chunk embeddings for similarity search.
- **Retriever:** Selects relevant chunks based on semantic similarity.
- **LLM Generator:** Produces the response using retrieved content.

These components are loosely coupled, making RAG flexible and easy to adapt to different domains.

◆ **Prompt Construction Pipeline**

Rather than feeding all knowledge to the LLM, RAG selects and injects only the **most relevant chunks** into the prompt alongside the user query.

This step is crucial to:

- Keep within token limits
- Ground the generation in facts
- Improve answer relevance

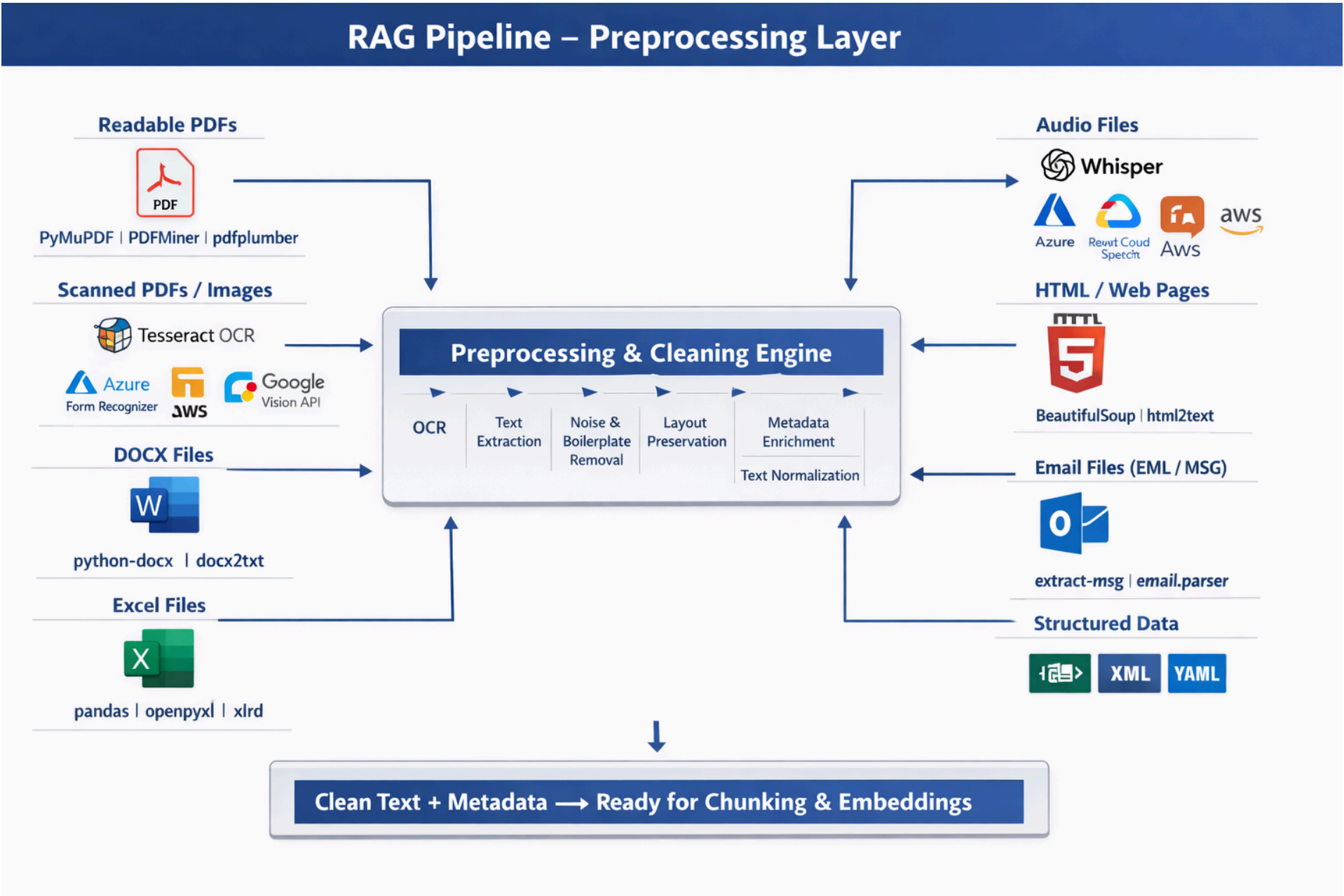
 Detailed strategies for chunking, retrieval ranking, and formatting prompts are covered in later sections.

4. Document Ingestion Pipeline

- Preprocessing
- Chunking Strategies
- Embedding and Storage
- Metadata Strategy and Schema Design

4.1 Preprocessing

Preprocessing is the critical first step in the RAG pipeline. It involves extracting and cleaning content from raw documents to prepare it for chunking, embedding, and retrieval. Since enterprise data comes in various formats, preprocessing must be tailored based on the document type.



✓ 1. Readable PDFs (Text-Based)

- Use libraries like **PyMuPDF**, **PDFMiner**, or **pdfplumber** to extract clean text.
- Remove watermarks, headers, footers, and footnotes if not useful.
- Preserve layout if structure is important (e.g., tables, bulleted lists).

✓ 2. Scanned PDFs (Image-Based)

- Apply **OCR (Optical Character Recognition)** using tools like:
 - **Tesseract OCR** (open-source)
 - **Azure Form Recognizer**, **AWS Textract**, or **Google Vision API** (for higher accuracy and layout retention)
- Post-process OCR output:
 - Fix common OCR artifacts (e.g., broken words, misread characters)
 - Validate text quality before passing it downstream

⚠ Use layout-aware OCR when dealing with forms, tables, or structured documents.

✓ 3. DOCX Files (Word Documents)

- Use libraries like **python-docx** or **docx2txt** to extract text.
- Optionally preserve styles (e.g., bold headings) for metadata tagging.

- Handle tracked changes, comments, or footnotes if present.

✔ 4. Excel Files

- Use **pandas**, **openpyxl**, or **xlrd** to extract tabular content.
- Convert tables into structured chunks (e.g., row-wise JSON records).
- Handle multiple sheets, hidden rows/columns, and merged cells.

✔ 5. Audio Files (e.g., Call Logs, Voice Notes)

- Apply **ASR (Automatic Speech Recognition)**:
 - Open-source: **Whisper by OpenAI**
 - Commercial: **Azure Speech-to-Text**, **Google Cloud Speech**, **AWS Transcribe**
- Clean and structure transcripts:
 - Separate speakers if needed
 - Remove filler words (“uh”, “um”) or background noise text
 - Use sentence segmentation if transcripts are long

✔ 6. HTML/Web Pages

- Use **BeautifulSoup** or **html2text** to extract content
- Remove ads, navigation, and boilerplate
- Retain important tags (like `<h1>` , `<p>`) for chunk boundary logic

✔ 7. Images with Text (e.g., screenshots, scanned images)

- Apply OCR tools (same as scanned PDFs)
- Use **layout-aware models** if the image contains structured layouts (e.g., invoices)

✔ 8. Email (EML/MSG files)

- Parse using **email.parser** or libraries like **extract-msg**
- Extract body, attachments, headers (sender, subject, timestamp)
- Optionally process inline attachments (e.g., embedded PDFs or images)

✔ 9. JSON, XML, YAML, and Structured Files

- Extract only required fields or sections
- Flatten nested content for consistency
- Remove irrelevant fields (like IDs, metadata noise)

⚙️ Post-Processing for All Formats

- Normalize whitespace and encoding
- Remove unwanted characters or boilerplate
- Add basic **document-level metadata** (filename, type, source, ingestion date)
- Validate final extracted text length and clarity before chunking

 A robust preprocessing pipeline ensures that garbage in → **does not become** garbage out. The cleaner your input, the more accurate your retrieval and generation.

◆ 4.2 Chunking

Chunking is the process of breaking large documents into smaller, semantically meaningful units (**chunks**) so they can be embedded, indexed, and retrieved efficiently in a Retrieval-Augmented Generation (RAG) system.

Chunking defines how knowledge is structured and accessed. Once data is chunked and embedded, retrieval quality is largely fixed.

🎯 Why Chunking Matters in RAG

Chunking directly impacts retrieval relevance, context efficiency, answer faithfulness, latency, and cost. Even strong LLMs hallucinate if retrieved chunks are poorly scoped.

✔ Why Chunking Is Required

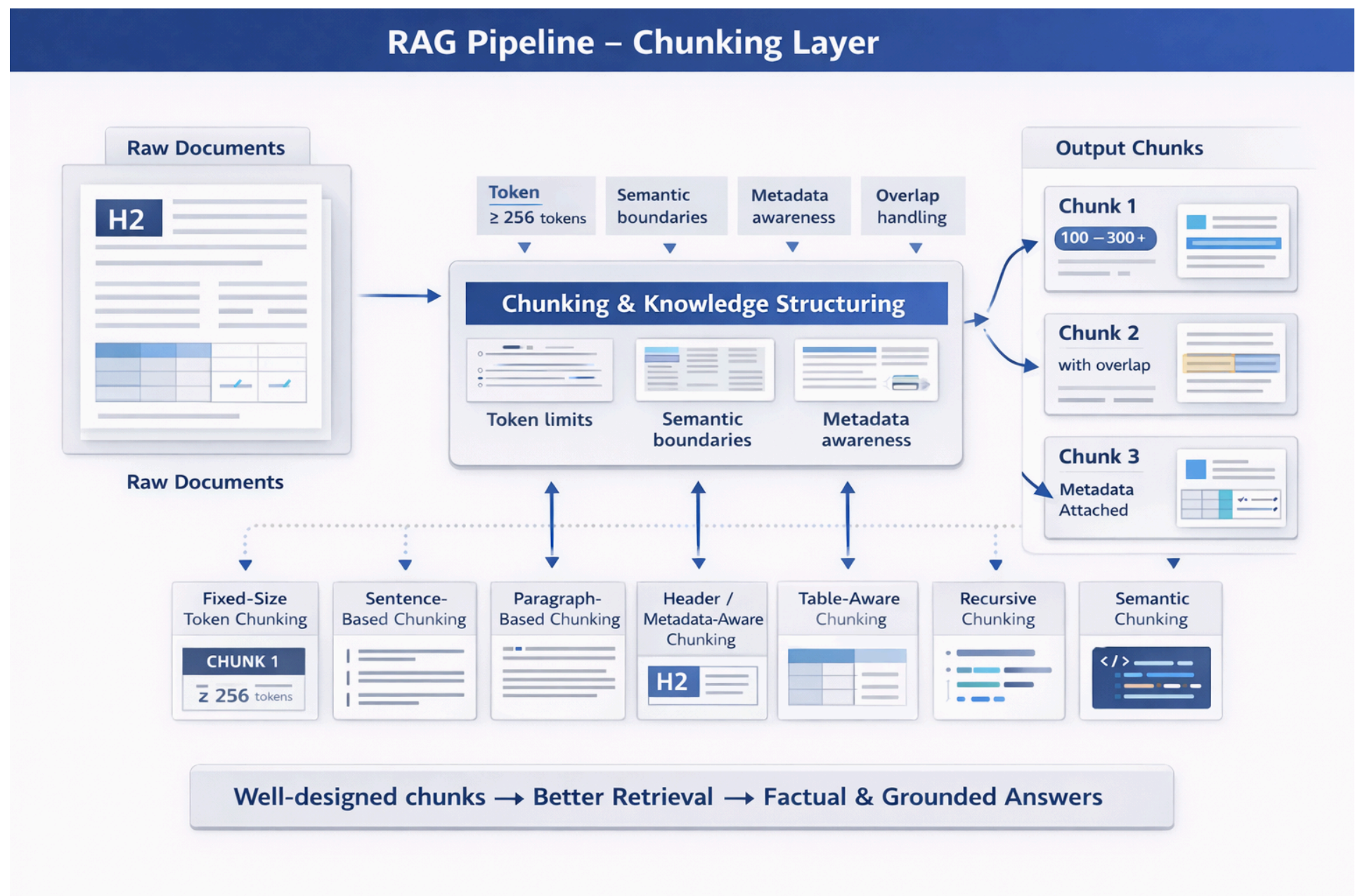
LLMs have limited context windows (4K–32K tokens), while real-world documents are large, unstructured, and often mixed-format. Chunking enables scalable indexing, fine-grained retrieval, reduced prompt noise, and cost-efficient inference.

✓ Chunk Size Considerations

Chunk size controls semantic density.

- Recommended: **100–300 tokens**
- Larger chunks → more context, lower precision
- Smaller chunks → higher precision, fragmented reasoning

Chunk size should align with embedding model capacity, document structure, and query style.



✓ Common Chunking Strategies

1. **Fixed-Size Token Chunking**
2. **Sentence-Based Chunking**
3. **Paragraph-Based Chunking**
4. **Header-Aware / Metadata-Aware Chunking**
5. **Table-Aware Chunking**
6. **Recursive Chunking**
7. **Semantic Chunking**
8. **Code-Aware Chunking**

◆ 1. Fixed-Size Token Chunking

Text is split into chunks of a fixed number of tokens, usually with overlap to preserve context across chunk boundaries. This approach prioritizes consistency and scalability over semantic boundaries.

Pros

- Simple and fast
- Predictable chunk sizes
- Works for any document type

Cons

- Ignores semantic structure

- May split sentences or ideas mid-way

```
def fixed_chunk(text, size=300, overlap=50):
    words = text.split()
    chunks = []
    for i in range(0, len(words), size - overlap):
        chunks.append(" ".join(words[i:i + size]))
    return chunks
```

When to use: Logs, plain text, fallback strategy

◆ **2. Sentence-Based Chunking**

Sentences are grouped together to form chunks, ensuring that each chunk contains complete linguistic units rather than arbitrary token splits.

Pros

- Better semantic alignment
- Preserves sentence integrity

Cons

- Uneven chunk sizes
- Not suitable for code or tables

```
import nltk
nltk.download("punkt")

def sentence_chunk(text, sentences_per_chunk=3):
    sentences = nltk.sent_tokenize(text)
    return [
        " ".join(sentences[i:i + sentences_per_chunk])
        for i in range(0, len(sentences), sentences_per_chunk)
    ]
```

When to use: Articles, blogs, FAQs, documentation text

◆ **3. Paragraph-Based Chunking**

Paragraph boundaries are used as chunk boundaries, preserving the author-defined grouping of ideas and logical flow.

Pros

- Preserves conceptual grouping
- Simple and intuitive

Cons

- Paragraphs may exceed token limits
- Short paragraphs may lack sufficient context

```
def paragraph_chunk(text):
    paragraphs = text.split("\n\n")
    return [p.strip() for p in paragraphs if p.strip()]
```

When to use: Legal documents, policies, HR manuals, contracts

◆ **4. Header-Aware / Metadata-Aware Chunking**

Content is grouped under section headers, and the header information is stored as metadata to maintain document hierarchy during retrieval.

Pros

- Strong semantic hierarchy
- Improved retrieval precision
- Enables section-level filtering

Cons

- Requires well-structured documents

```
def header_aware_chunk(sections):
    chunks = []
    for header, content in sections.items():
        chunks.append({
            "text": content,
            "metadata": {"section": header}
        })
    return chunks
```

When to use: SOPs, policy documents, user manuals, academic reports

◆ 5. Table-Aware Chunking

Tables are processed separately from surrounding text to preserve their structured nature, with headers retained and rows grouped logically.

Pros

- Preserves tabular structure
- Improves accuracy for numeric data

Cons

- Requires special parsing logic
- Not directly compatible with sentence-based methods

```
def table_chunk(rows, headers):
    return [
        {"text": dict(zip(headers, row))}
        for row in rows
    ]
```

When to use: Financial documents, Excel exports, tabular PDFs

◆ 6. Recursive Chunking (Highly Recommended)

Chunking is applied in multiple passes, attempting to preserve document structure first and falling back to fixed-size splitting only when necessary.

Pros

- Preserves semantic hierarchy
- Balances structure and scalability

Cons

- Slightly more complex implementation

```
def recursive_chunk(text, max_tokens=300):
    paragraphs = text.split("\n\n")
    chunks = []
    for p in paragraphs:
        if len(p.split()) <= max_tokens:
            chunks.append(p)
        else:
            chunks.extend(fixed_chunk(p, max_tokens))
    return chunks
```

When to use: Mixed-format documents, PDFs, research papers

◆ 7. Semantic Chunking (Advanced)

Chunk boundaries are determined using embedding similarity, grouping sentences until a semantic shift is detected.

Pros

- Topic-coherent chunks
- High retrieval precision

Cons

- Higher preprocessing cost
- More complex pipeline

```
# Conceptual pseudo-code
if similarity(sentence_i, sentence_j) > threshold:
    same_chunk.append(sentence_j)
else:
    start_new_chunk()
```

When to use: High-accuracy RAG systems, compliance or regulatory use cases

◆ 8. Code-Aware Chunking

Code is chunked based on structural units such as functions and classes, preserving indentation, comments, and logical flow.

Pros

- Preserves code semantics
- Improves retrieval accuracy for developer queries

Cons

- Requires language-aware parsing

```
def code_chunk(functions):  
    return [  
        {"text": func["code"], "metadata": {"name": func["name"]}}  
        for func in functions  
    ]
```

When to use: Developer documentation, code assistants, API reference RAG

◆ 4.3 Text Embedding (RAG – Data Ingestion Layer)

Once documents are chunked, the next step in the RAG ingestion pipeline is to **convert each chunk into a numerical vector** using an embedding model. These vectors, called **embeddings**, are stored in a **vector database** and represent the semantic meaning of each chunk.

This stage defines **how knowledge is represented** inside the RAG system. Retrieval quality later depends heavily on how embeddings are generated and stored.

Purpose of Embeddings in RAG Ingestion

Embeddings enable:

- Semantic representation of text chunks
- Efficient similarity-based retrieval
- Scalable storage of knowledge
- Model-agnostic grounding for LLM responses

Embeddings capture **meaning**, not keywords.

What Is an Embedding?

An **embedding** is a **fixed-length dense numerical vector** that represents the **semantic meaning** of a text chunk.

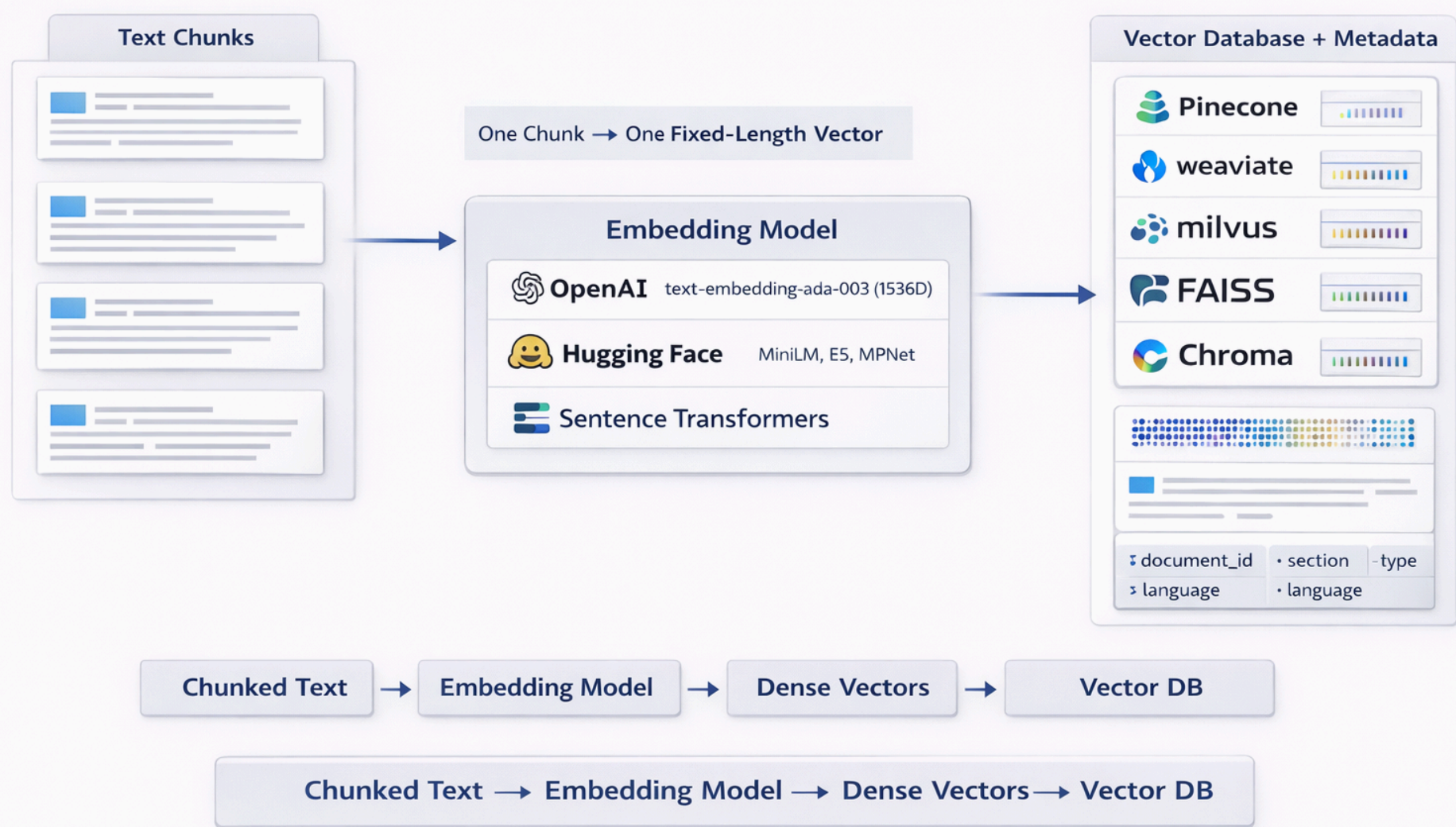
Key properties:

- Same-sized vector for every chunk
- Similar meanings → vectors closer in space
- Different wording with same intent → similar embeddings
- Independent of document length or formatting

For **text-embedding-ada-003** :

- Output size = **1536 dimensions**
 - One chunk → one 1536-value array
-

RAG Pipeline – Text Embedding Layer (Data Ingestion)



Embedding Flow (Ingestion Side Only)

```
Chunk (text)
  ↓
Embedding Model (ada-003)
  ↓
N-dimensional vector
  ↓
Vector Database + Metadata
```

No additional processing layers are exposed to the user.

◆ One Chunk = One Embedding

In RAG ingestion:

- ✗ No word-level embeddings
- ✗ No token-level embeddings
- ✗ No sentence-level embeddings (unless sentence is the chunk)
- ✓ **Exactly one embedding per chunk**

Number of chunks = number of embeddings stored.

◆ Example: Embedding a Single Chunk

Chunk Text

"Employee data must not be shared over unauthorized channels."

Output Embedding (Conceptual)

[0.021, -0.113, 0.487, 0.092, ..., -0.044] ← 1536 values

This vector becomes the **semantic identity** of the chunk inside the vector database.

◆ Why Chunking Must Happen Before Embedding

- Embeddings represent **one semantic unit**
- Large chunks dilute meaning
- Small, coherent chunks produce higher-quality embeddings
- Embedding quality cannot be fixed later

Chunking defines **what the embedding represents**.

◆ Embedding Models Commonly Used in RAG

Model Name	Dimensions	Notes
text-embedding-ada-003	1536	OpenAI, production-grade
all-MiniLM-L6-v2	384	Lightweight, fast
bge-small-en	384	Open-source, instruction-tuned
e5-base	768	Strong for QA-style retrieval
paraphrase-mpnet-base-v2	768	High semantic accuracy

Choose based on **accuracy, latency, cost, and scale**.

◆ What Is Stored in the Vector Database

Each embedded chunk is stored with metadata:

```
{
  "id": "chunk_045",
  "embedding": [1536 values],
  "text": "Employees may apply for leave after probation.",
  "metadata": {
    "document_id": "HR_policy_2023",
    "section": "5.3",
    "doc_type": "leave_policy",
    "language": "en"
  }
}
```

✓ Best Practices for Embedding (Ingestion Layer)

1. Clean text before embedding (OCR errors, whitespace)
2. Use consistent chunking strategy
3. Batch embedding requests for efficiency
4. Store rich but minimal metadata
5. Track embedding model version
6. Avoid duplicate chunks
7. Keep embeddings immutable after ingestion
8. Separate indexes for unrelated domains if needed

Working of Ada-003 Embedding Model

1. What is Ada-003

Ada-003 is an **encoder-only Transformer model** used to convert text into **dense semantic embeddings**.

- Purpose: semantic understanding, not generation
- Output: one vector per input text
- Final embedding size: **1536 dimensions**

2. What Ada-003 Does Not Do

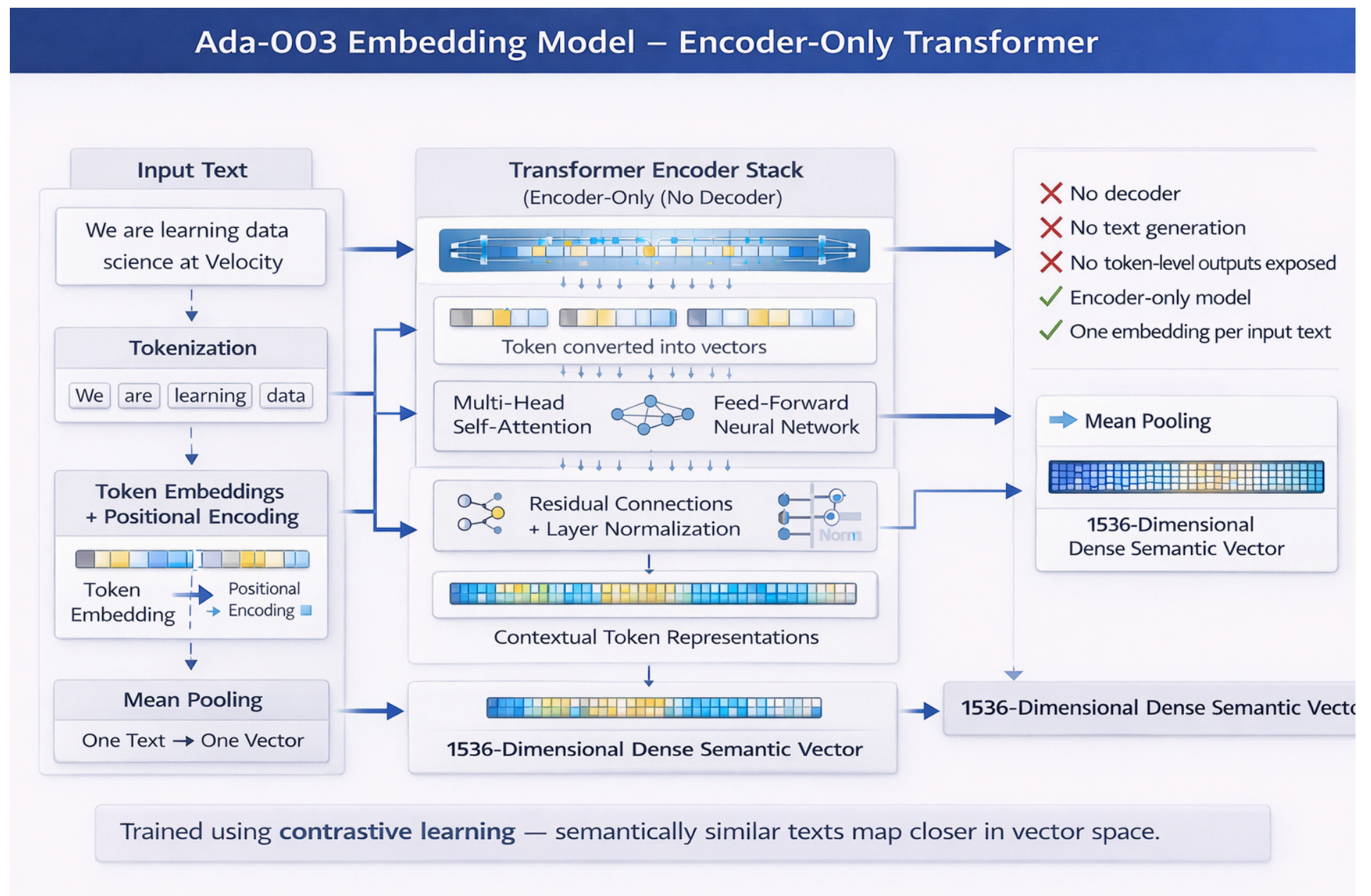
- No text generation
- No decoder stack
- No next-token prediction

- No token-level output exposed to users

3. High-Level Architecture Flow

Text

- Tokenization
- Token Embeddings + Positional Encoding
- Transformer Encoder Stack
- Contextual Token Representations
- Mean-style Pooling
- (Optional) Linear Projection
- Final 1536-D Embedding



4. Encoder Stack

Ada-003 uses **only Transformer encoders**.

Each encoder layer contains:

- Multi-head self-attention
- Feed-forward neural network
- Residual connections and layer normalization

Self-attention is **bidirectional**, so each token attends to all others.

5. Token-Level Representations

After the final encoder layer:

- Each token has a **context-aware vector**
- Each token vector already encodes information from the full sentence

OpenAI does **not** publicly disclose the internal token hidden size.

6. Sentence-Level Representation (Mean Pooling)

Downstream systems require **one vector per text**, so token vectors are aggregated.

If the encoder outputs:

v1, v2, v3, ..., vn

Then the sentence embedding is:

(v1 + v2 + v3 + ... + vn) / n

Mean pooling works well because:

- Context is already captured by self-attention
- Stopwords naturally contribute less
- Embeddings are stable across sentence lengths

7. Linear Projection

After pooling, a **linear projection layer may be applied**.

Important points:

- Projection is not only for changing dimensions
- It helps shape embedding geometry for similarity tasks
- Common in contrastive learning models
- May exist even if input and output dimensions are both 1536

OpenAI does not confirm whether this layer exists internally.

8. Training Objective

Ada-003 is trained using **contrastive learning**.

- Semantically similar texts → embeddings closer
- Semantically different texts → embeddings farther apart

Similarity is typically measured using **cosine similarity**.

9. Detailed Example

Input sentence:

We are learning data science at Velocity

Tokenized as:

["We", "are", "learning", "data", "science", "at", "Velocity"]

After the encoder stack, each token has a contextual vector (example shown in 4 dimensions for simplicity):

Token	Vector
We	[0.1, 0.1, 0.1, 0.1]
are	[0.0, 0.0, 0.0, 0.0]
learning	[1.0, 0.9, 0.8, 0.7]
data	[1.1, 1.0, 0.9, 0.8]
science	[1.1, 1.0, 1.0, 0.9]
at	[0.0, 0.0, 0.0, 0.0]
Velocity	[1.2, 1.1, 1.0, 0.9]

Mean pooling:

Sum:

[4.5, 4.1, 3.8, 3.4]

Divide by number of tokens (7):

[0.64, 0.59, 0.54, 0.49]

This vector represents the **full meaning of the sentence**. In the real model, this would be a **1536-dimensional vector**.

10. Semantic Behavior

Sentence:

We are learning data science at Velocity

Another sentence:

Velocity students are studying data science

Both produce embeddings that are **very close in vector space**, despite different wording.

11. Implementation

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

text = "We are learning data science at Velocity"

resp = client.embeddings.create(
    model="text-embedding-3-small", # or "text-embedding-3-large"
    input=text
)

vec = resp.data[0].embedding
print("Embedding length:", len(vec))
print("First 8 values:", vec[:8])
```

12. Key Facts to Remember

- Encoder-only Transformer
 - One embedding per text
 - Final embedding size = 1536
 - Mean-style pooling used
 - Optional projection head possible
 - Internal token size not publicly disclosed
-
-

4.4 Vector Databases

Vector databases are the backbone of RAG systems. They store high-dimensional embeddings and enable efficient similarity search. Many also support metadata filtering, hybrid search, and horizontal scalability across millions of vectors.

During retrieval, embedded queries are compared against stored embeddings using similarity metrics such as cosine similarity or dot product.

1. FAISS

- Local, open-source ANN library by Meta
- Excellent for development, evaluation, and on-prem PoCs
- Lacks built-in metadata filtering and persistence

```
import faiss
import numpy as np

dim = 1536
index = faiss.IndexFlatL2(dim)

index.add(np.array([embedding_1, embedding_2]).astype("float32"))
D, I = index.search(np.array([query_embedding]).astype("float32"), k=2)
Best for: Prototyping and research environments
```

2. Pinecone

- Fully managed, cloud-native vector database
- Highly scalable with metadata filtering and streaming inserts
- Supports hybrid (sparse + dense) search


```
import pinecone

pinecone.init(api_key="API_KEY", environment="us-east1-gcp")
index = pinecone.Index("rag-index")

index.upsert([
    ("chunk_1", embedding_1, {"doc": "HR_policy"})
])

index.query(vector=query_embedding, top_k=3, include_metadata=True)
```

Best for: Production-grade systems with minimal operational overhead

3. Weaviate

- Open-source and hosted options available
- RESTful API with metadata filtering and hybrid search
- Schema-driven and customizable

```
import weaviate

client = weaviate.Client("http://localhost:8080")

client.data_object.create(
    data_object={"text": chunk_text},
    class_name="Document",
    vector=embedding
)

client.query.get("Document", ["text"]) \
    .with_near_vector({"vector": query_embedding}) \
    .with_limit(3).do()
```

Best for: Customizable and scalable enterprise deployments

4. Chroma

- Lightweight and simple to integrate
- Local storage with no server overhead
- Supports basic metadata filtering

```
import chromadb

client = chromadb.Client()
collection = client.create_collection("rag_docs")

collection.add(
    documents=[chunk_text],
    embeddings=[embedding],
    metadatas=[{"source": "policy"}],
    ids=["chunk_1"]
)

collection.query(query_embeddings=[query_embedding], n_results=3)
```

Best for: Rapid prototyping, educational projects, or small-scale RAG

5. Milvus

- Distributed, cloud-ready vector database
- High throughput and concurrency support
- Supports scalar filtering and hybrid indexes

```
from pymilvus import Collection

collection = Collection("rag_collection")

collection.insert([
    [embedding],
    ["chunk_1"]
])

collection.search(
    data=[query_embedding],
    anns_field="embedding",
    param={"metric_type": "L2"},
    limit=3
)
```


Best for: Enterprise-grade RAG systems at scale

6. Azure AI Search

- Fully managed search platform with vector indexing
- Supports dense and sparse hybrid search
- Tight integration with Azure ecosystem

```
from azure.search.documents import SearchClient

client = SearchClient(endpoint, index_name, credential)

client.upload_documents([
    {"id": "chunk_1", "embedding": embedding, "content": chunk_text}
])

client.search(search_text=None, vector=query_embedding, top_k=3)
```

Best for: Azure-native enterprise RAG pipelines

7. Amazon OpenSearch (k-NN)

- AWS-native search engine with vector search support
- Enables hybrid keyword and vector retrieval
- Integrates with AWS services like S3 and Bedrock

```
from opensearchpy import OpenSearch

client = OpenSearch(hosts=[{"host": "localhost", "port": 9200}])

client.index(
    index="rag-index",
    body={"embedding": embedding, "text": chunk_text}
)

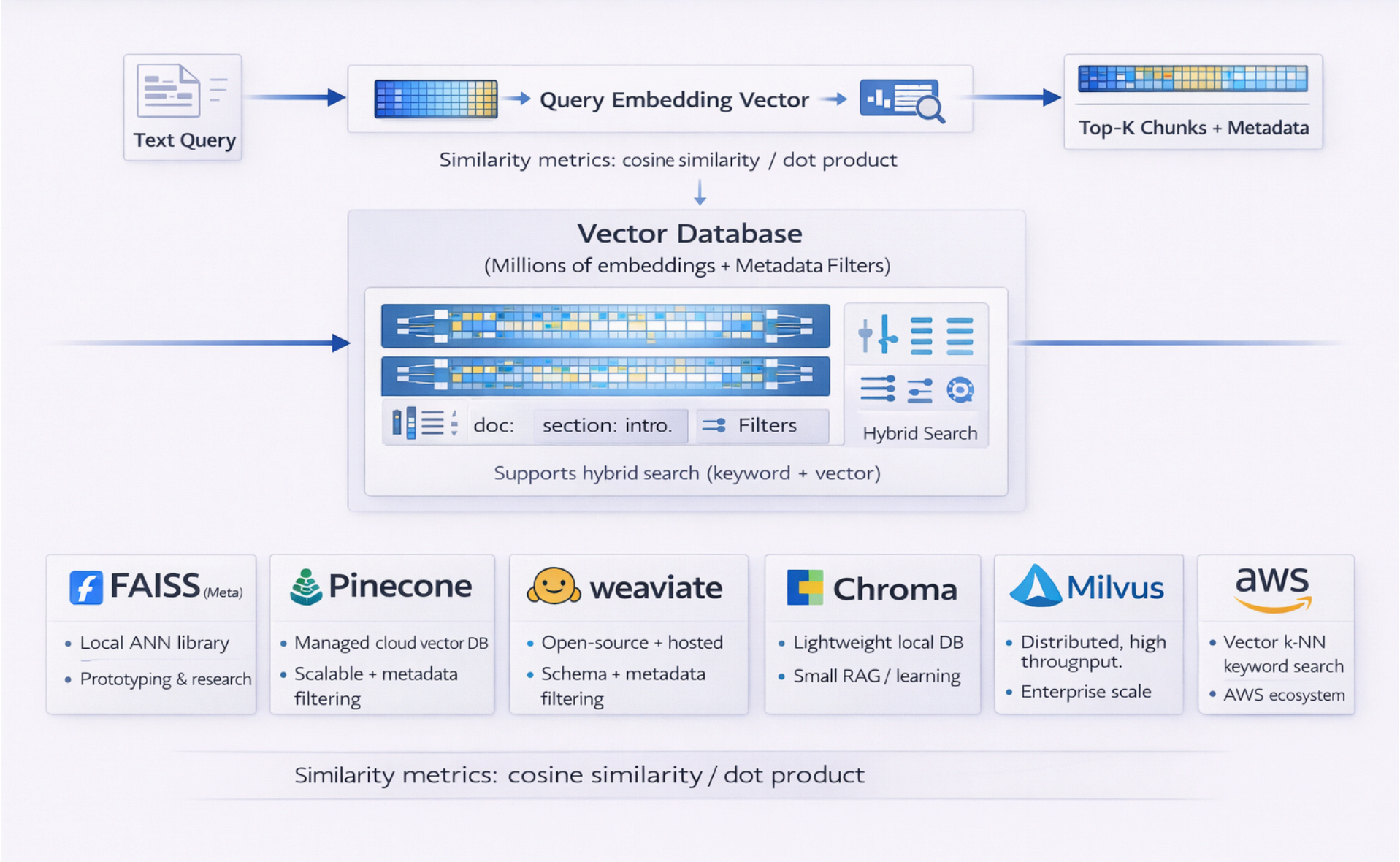
client.search(
    index="rag-index",
    body={"knn": {"embedding": {"vector": query_embedding, "k": 3}}}
)
```

Best for: AWS-native hybrid RAG applications

Vector Database Comparison (RAG – Practical View)

Vector DB	Deployment Type	Metadata Filtering	Hybrid Search	Available on AWS	Available on Azure
FAISS	Local / Library	No	No	Yes	Yes
Pinecone	Managed SaaS	Yes	Yes	Yes	Yes
Weaviate	Cloud / On-prem	Yes	Yes	Yes	Yes
Chroma	Local	Basic	No	Yes	Yes
Milvus	Distributed	Yes	Yes	Yes	Yes
Azure AI Search	Cloud (PaaS)	Yes	Yes	No	Yes (Native)
Amazon OpenSearch	Cloud (Managed)	Yes	Yes	Yes (Native)	No

Vector Databases in RAG Systems



◆ 4.5 Metadata Strategy and Schema Design

While embeddings power semantic retrieval, **metadata** acts as the control layer that makes RAG systems scalable, secure, and context-aware. A robust metadata strategy is critical for enabling **filtering**, **access control**, **domain separation**, **compliance**, and **retrieval optimization**.

✓ Why Metadata Matters in RAG

In a production RAG pipeline, chunks alone are not enough. You often need to:

- Retrieve only chunks from a specific **document type**, **section**, or **client**
- Isolate documents per **region**, **user group**, or **tenant**
- Perform **filtered search** before applying vector similarity
- Enforce **data access control policies**
- Enable **auditing** and **explainability** (e.g., "this answer came from Plan X, Section 8.5")

✓ Common Metadata Fields

When storing each chunk in the vector database, you should associate it with structured metadata fields such as:

Field Name	Description
document_id	Unique identifier of the source document
document_type	e.g., retirement_policy , health_form , leave_policy
client	e.g., Fidelity , Sun Life , Manulife
section	Section or clause reference (e.g., 8.1.2)
plan_id	Specific plan identifier (useful in insurance or retirement documents)
language	Document language (e.g., en , fr , es)
source_type	PDF, DOCX, Email, Audio, Web, etc.
access_level	e.g., public , internal , restricted , HR-only
ingestion_date	Timestamp of when this chunk was last updated

✦ Choose metadata based on your domain — legal, HR, finance, healthcare, etc.

✓ Example Chunk with Metadata

```
{
  "id": "chunk_01234",
  "text": "Employees may carry forward up to 10 unused leave days to the next year.",
  "embedding": [...],
  "metadata": {
    "document_id": "leave_policy_2024",
    "client": "Sun Life",
    "section": "5.2",
    "plan_id": "SLP-457B",
    "document_type": "leave_policy",
    "language": "en",
    "access_level": "internal",
    "ingestion_date": "2024-12-10T14:00:00Z"
  }
}
```

This metadata enables:

- **Filtered retrieval** (e.g., only `leave_policy` documents for `Sun Life`)
- **Source traceability** (e.g., chunk came from Section 5.2)
- **Access restriction enforcement** (e.g., HR-only chunks)

✓ Designing a Metadata Schema

Treat metadata schema design like API schema design — it must be:

- **Consistent** across all document types
- **Extensible** (e.g., new fields can be added without breaking old queries)
- **Queryable** (optimized for vector DB filtering)
- **Typed** (e.g., `plan_id` should always be a string or integer, not mixed)

💡 Pro Tip: Keep **field names lowercase** and avoid special characters or spaces to ensure compatibility with most vector DBs (e.g., Pinecone, Weaviate, FAISS + custom filters).

✓ Best Practices

1. **Always store metadata per chunk**, even if redundant across a document
2. **Use metadata filtering before similarity search** to narrow search scope
3. **Version document ingestion**, especially for legal/compliance use cases
4. **Avoid hardcoding access logic** — use metadata like `access_level` or `department` to apply role-based filters dynamically
5. **Auditability**: Store `ingestion_date`, `source_type`, and `document_id` to trace any response back to the original source
6. **Avoid excessive metadata nesting** — keep metadata flat for better performance and compatibility

✓ Use Cases Enabled by Metadata

Use Case	Metadata Involved
Client-specific HR assistant	<code>client</code> , <code>document_type</code>
Regulatory-compliant Q&A	<code>access_level</code> , <code>document_id</code> , <code>section</code>
Multilingual assistant	<code>language</code>
Plan-level policy search	<code>plan_id</code> , <code>section</code>
Audit trail of responses	<code>ingestion_date</code> , <code>document_id</code>
Scoped retrieval (e.g., U.S. only)	<code>region</code> or <code>jurisdiction</code> (custom field)

✓ In production-grade RAG systems, well-designed metadata is **as important as good embeddings** — it drives retrieval precision, speed, access control, and explainability.

5. Retrieval (RAG Core Layer)

Retrieval is the step in a RAG system where the retriever selects the most relevant **document chunks** from storage (vector DB and sometimes keyword index) to answer a user query. The selected chunks become the **grounding context** for the LLM.

If retrieval returns the wrong chunks, the LLM is forced to guess. That is why retrieval quality controls:

- correctness (right facts)
 - completeness (enough coverage)
 - faithfulness (low hallucination)
 - efficiency (less noise, fewer tokens)
-

5.0 Retrieval Techniques – Overview

This retrieval section covers the following strategies:

1. Top-K Similarity Retrieval
 2. Maximal Marginal Relevance (MMR)
 3. Metadata-Filtered Retrieval
 4. Self-Query Retrieval
 5. Hybrid Retrieval (Dense + Sparse)
-

5.1 Where Retrieval Fits in the RAG Pipeline

Retrieval happens after ingestion is complete (chunking + embeddings + vector DB indexing).

Documents → Chunking → Embeddings → Vector DB → Retrieval → Re-ranking → Context Assembly → LLM

Retrieval is a gatekeeper: it decides what knowledge the LLM can access.

5.2 Top-K Similarity Retrieval

Top-K retrieval returns the **K most similar chunks** to the query, based on a similarity function (cosine similarity or dot product). This is the baseline retriever used in most RAG systems.

How it works (what actually happens)

1. Query is embedded into a vector (same embedding model space used for documents)
2. Vector DB searches for nearest neighbors among stored chunk vectors
3. Top-K results are returned with chunk text + metadata + similarity scores

Why it works

When chunking is clean and embeddings are high-quality, the nearest vectors usually represent the best semantic matches, even when user wording differs from the document wording.

Pros

- Fast and easy to implement
- Scales well (ANN indexes)
- Strong baseline in production

Cons

- Redundancy: top results can come from the same paragraph/section
- Low diversity: may miss secondary supporting context
- Can struggle with jargon/acronyms if documents rely heavily on exact terms (hybrid helps here)

When to use

- Clean corpora with good chunking
- Straightforward Q&A
- First version / baseline retriever

Small code snippet (generic search)

```
results = vector_db.search(
    query_embedding=query_embedding,
    top_k=5,
    filter={"doc_type": "policy"},    # optional
    include_metadata=True
)
```

Small code snippet (Chroma)

```
out = collection.query(
    query_embeddings=[query_embedding],
    n_results=5,
    where={"doc_type": "leave_policy"}
```

```
)
docs = out["documents"][0]
```

5.3 Maximal Marginal Relevance (MMR)

MMR improves retrieval by balancing:

- relevance to the query
- diversity among selected chunks

Instead of picking the top-K most similar chunks (which may be repetitive), MMR picks chunks that add **new information** while staying relevant.

What problem it solves

If Top-K returns:

- “Section 5.1 leave eligibility”
- “Section 5.1 leave eligibility (rephrased)”
- “Section 5.1 leave eligibility (again)”

MMR tries to return something like:

- eligibility rules
- carry forward rules
- approval workflow
- exceptions/edge cases

How it works (high-level)

1. Retrieve a larger candidate set (e.g., top-20 or top-50)
2. Select chunks iteratively:
 - choose one that is relevant to query
 - penalize chunks that are too similar to already-selected ones

Pros

- Removes redundancy
- Adds coverage for broad queries
- Very useful for policy/legal corpora

Cons

- Slightly slower (re-ranking loop)
- Needs tuning of λ (relevance vs diversity trade-off)

When to use

- Repetitive documents (policies, manuals)
- Broad queries needing coverage across multiple sections
- When Top-K results look too similar

Small code snippet (MMR with LangChain-style idea)

```
# conceptual: retrieve many, then diversify
candidates = vector_db.search(query_embedding, top_k=25)
final_docs = mmr_select(query_embedding, candidates, k=5, lambda_=0.7)
```

Small code snippet (minimal MMR selection skeleton)

```
def mmr_select(query_vec, candidates, k=5):
    selected = []
    for _ in range(k):
        best = max(candidates, key=lambda c: c["score"]) # simplified
        selected.append(best)
        candidates.remove(best)
    return selected
```

(Implementation detail: real MMR penalizes similarity to already-selected chunks.)

5.4 Metadata-Filtered Retrieval

Metadata-filtered retrieval applies structured filters (client, doc_type, year, language, region) before vector similarity search.

Why this is important

Without filtering, a query like “leave policy” might retrieve:

- HR leave policy chunks
- insurance claim policy chunks
- legal compliance leave clauses from another client

Filters prevent cross-domain noise and ensure only the correct subset of the corpus is searched.

Pros

- Big precision improvement
- Essential for enterprise/multi-tenant systems
- Reduces irrelevant chunk pollution

Cons

- Depends on accurate metadata tagging
- Over-filtering may hide relevant results if metadata is missing or wrong

When to use

- Enterprise systems (multi client, multi document types)
- Regulated environments (HR, legal, finance, insurance)

Small code snippet (Pinecone-style filtering)

```
results = index.query(  
    vector=query_embedding,  
    top_k=5,  
    include_metadata=True,  
    filter={"client": {"$eq": "SunLife"}, "doc_type": {"$eq": "leave_policy"}}  
)
```

Small code snippet (Chroma filter)

```
out = collection.query(  
    query_embeddings=[query_embedding],  
    n_results=5,  
    where={"client": "SunLife", "doc_type": "leave_policy"}  
)
```

5.5 Self-Query Retrieval

Self-query retrieval uses an LLM to interpret the user query and generate **structured metadata filters** automatically.

This is helpful when users don’t explicitly specify metadata but the query implies it.

Example

User: “Leave policy for SunLife in 2024”

LLM extracts:

- client = SunLife
- year = 2024
- doc_type = leave_policy

Then retrieval runs with those filters.

Pros

- Automatically applies correct filtering
- Strong for large schemas with many metadata fields
- Reduces irrelevant retrieval in enterprise environments

Cons

- Extra LLM call adds latency and cost
- Needs consistent metadata design
- Must be guarded to avoid hallucinated filters

When to use

- Multi-tenant enterprise RAG
- Strong metadata discipline
- Queries that commonly omit fields

Small code snippet (LLM produces filters)

```
def extract_filters(llm, query):
    prompt = f"Extract filters as JSON (client, doc_type, year): {query}"
    return llm(prompt)
```

Small code snippet (apply filters to retrieval)

```
filters = extract_filters(llm, user_query)
results = index.query(vector=query_embedding, top_k=5, filter=filters)
```

5.6 Hybrid Retrieval (Dense + Sparse)

Hybrid retrieval combines:

- dense retrieval (embeddings, semantic)
- sparse retrieval (BM25 / keyword)

Hybrid works better because:

- dense handles paraphrases and meaning
- sparse handles exact terms, acronyms, IDs, legal clauses

Pros

- Best real-world accuracy in many corpora
- Handles jargon/acronyms/IDs better
- Improves recall on technical and legal data

Cons

- More pipeline complexity
- Needs fusion / weighting / re-ranking

When to use

- Technical documents, compliance/legal text
- Queries with exact terms, codes, abbreviations
- Large heterogeneous datasets

Small code snippet (conceptual hybrid merge)

```
dense = dense_search(query_embedding, top_k=10)
sparse = bm25_search(user_query, top_k=10)
merged = fuse(dense, sparse, alpha=0.6)
```

Small code snippet (simple fusion)

```
def fuse(dense, sparse, alpha=0.6):
    scores = {}
    for r in dense:
        scores[r["id"]] = scores.get(r["id"], 0) + alpha * r["score"]
    for r in sparse:
        scores[r["id"]] = scores.get(r["id"], 0) + (1 - alpha) * r["score"]
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

5.7 Re-ranking (Post-Retrieval Precision Layer) ★

Re-ranking is a **post-retrieval refinement step** that reorders candidate chunks using a **cross-encoder model**.

What Re-ranking Does

Re-ranking evaluates each **(query, chunk)** pair together:

[Query + Chunk] → Cross-Encoder → Relevance Score

This allows:

- token-level interaction
 - intent understanding
 - answer completeness detection
-

Why Retrieval and Re-ranking Are Both Needed

Aspect	Candidate Retrieval (Bi-encoder)	Re-ranking (Cross-encoder)
Pipeline role	Candidate selection	Final ordering
Encoding	Query & chunk encoded separately	Query & chunk encoded jointly
Interaction	None	Full token-level attention
Speed	Very fast	Slow
Optimization goal	Recall	Precision

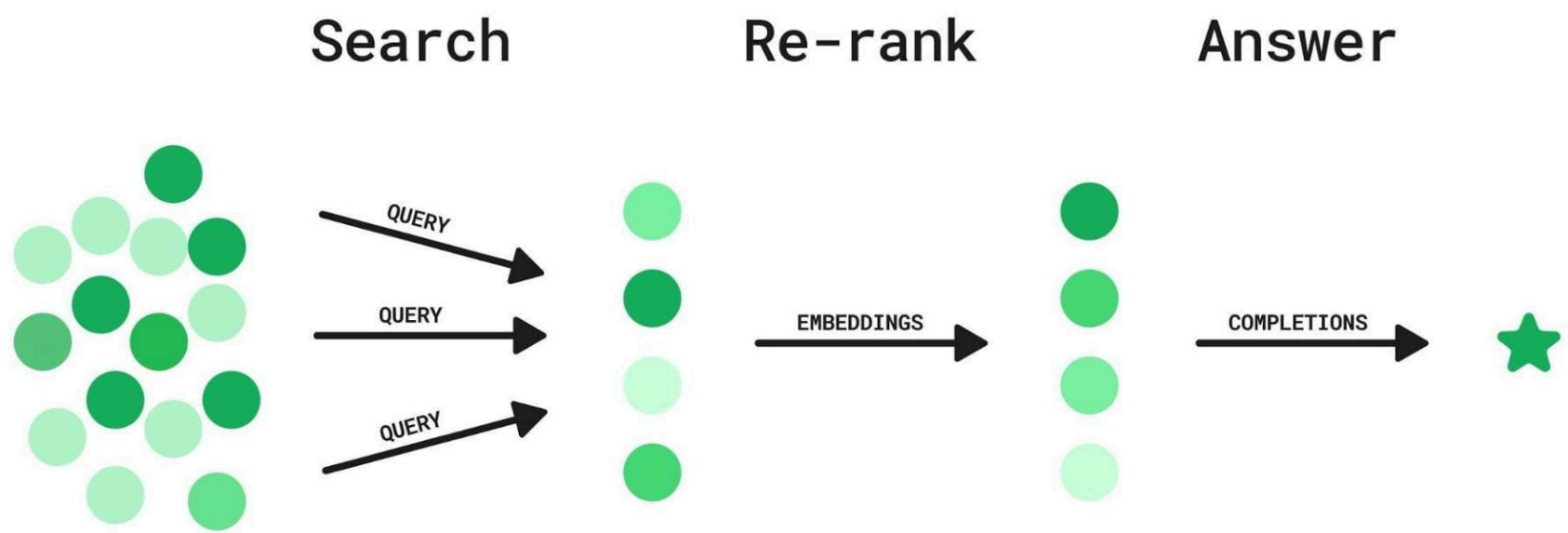
✦ This table compares capability, not execution order.

What Re-ranking Solves

- Removes weak or misleading chunks
- Promotes chunks that truly answer the question
- Reduces hallucination risk

When to Use Re-ranking

- Production RAG systems
- HR, legal, finance, compliance use cases
- Any system where **answer correctness matters**



5.8 Retrieval Strategy Comparison

Strategy	Core Purpose	Operates On	Key Benefit
Top-K	Recall	Corpus	Fast relevance
MMR	Diversity	Candidates	Less redundancy
Metadata Filter	Precision	Corpus scope	Domain safety
Self-Query	Automation	Metadata	Correct scoping
Hybrid	Recall boost	Corpus	Keyword + semantic
Re-ranking	Precision	Candidates	Final answer quality

5.9 Best Practices

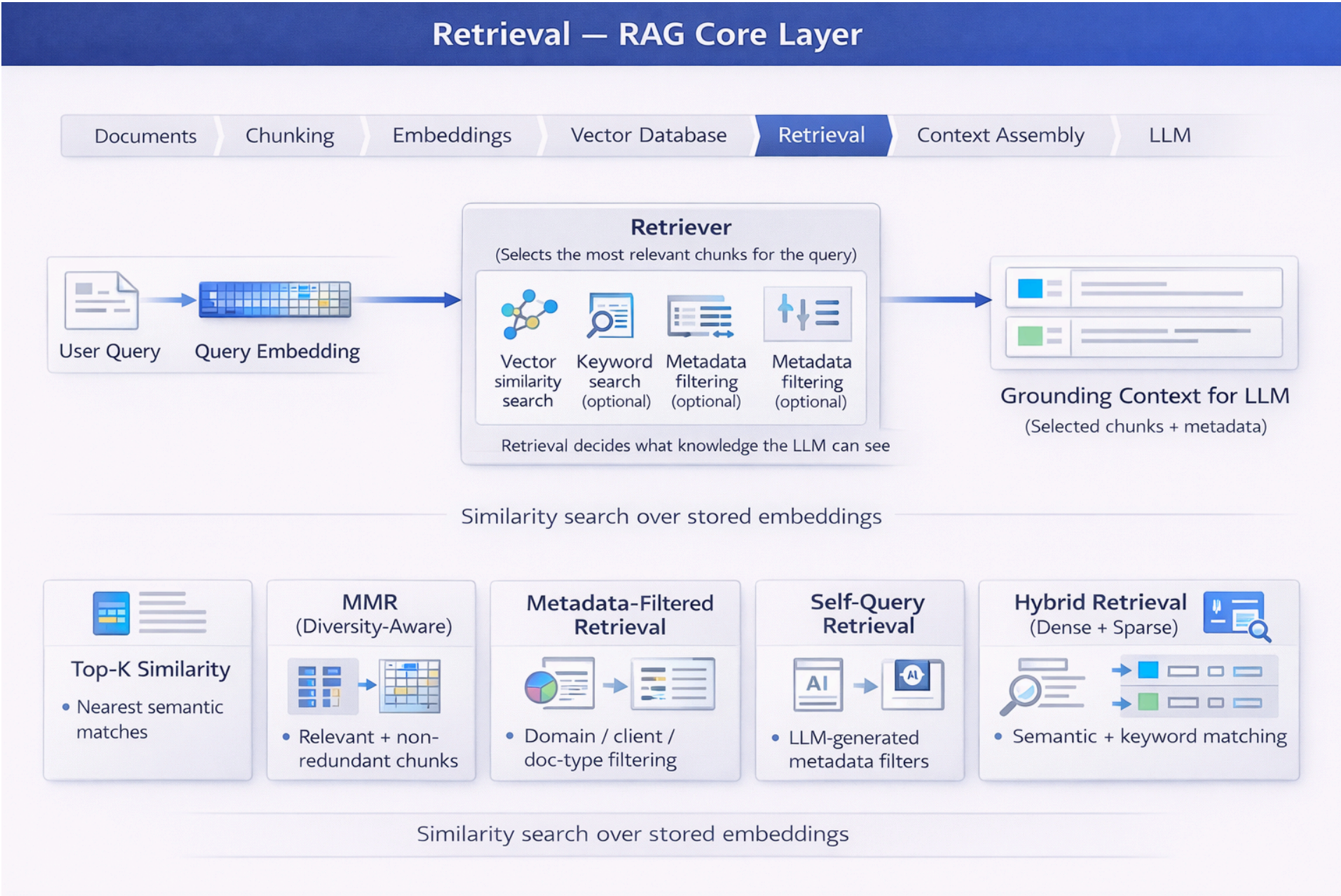
1. Start with dense Top-K as baseline
2. Apply metadata filtering wherever possible
3. Use MMR when results are repetitive
4. Use hybrid retrieval for jargon-heavy corpora
5. **Always apply re-ranking before LLM generation in production**
6. Prefer fewer high-quality chunks
7. Evaluate retrieval and re-ranking separately

5.10 Common Failure Modes

- High cosine score but wrong answer chunk
- Redundant chunks dominating Top-K
- Missing critical chunk (low recall)
- Correct chunk retrieved but poorly ordered
- No re-ranking → weak chunk fed to LLM

Conclusion

Cosine similarity is used for scalable candidate retrieval (recall), while re-ranking is used afterward for precise relevance judgment (precision). They are complementary layers, not competing techniques.



Detailed explanation of Re-Ranking in Retrieval-Augmented Generation (RAG)

Re-ranking is a critical stage in a RAG pipeline where the system **re-orders the initially retrieved document chunks using a deep semantic relevance model** before passing them to the language model for answer generation.

Vector search is optimized for **speed and recall**, not precision. It retrieves chunks that are semantically related to the query, but it cannot reliably decide:

- whether the chunk directly answers the question
- whether conditions and scope match the user's intent
- whether exceptions or exclusions apply

Re-ranking solves this by performing **fine-grained semantic reasoning** between the query and each retrieved chunk.

Why Re-Ranking Is a Separate Stage

Re-ranking is intentionally separated from retrieval because:

- embedding similarity is approximate
- deep semantic comparison is computationally expensive

- applying a heavy model to the entire corpus is infeasible

Hence, modern RAG systems follow:

- **retrieve broadly**
 - **re-rank precisely**
 - **generate confidently**
-

Cross-Encoder Re-Ranking (Most Common Approach)

The most widely used re-ranking approach is **cross-encoder re-ranking**.

A cross-encoder:

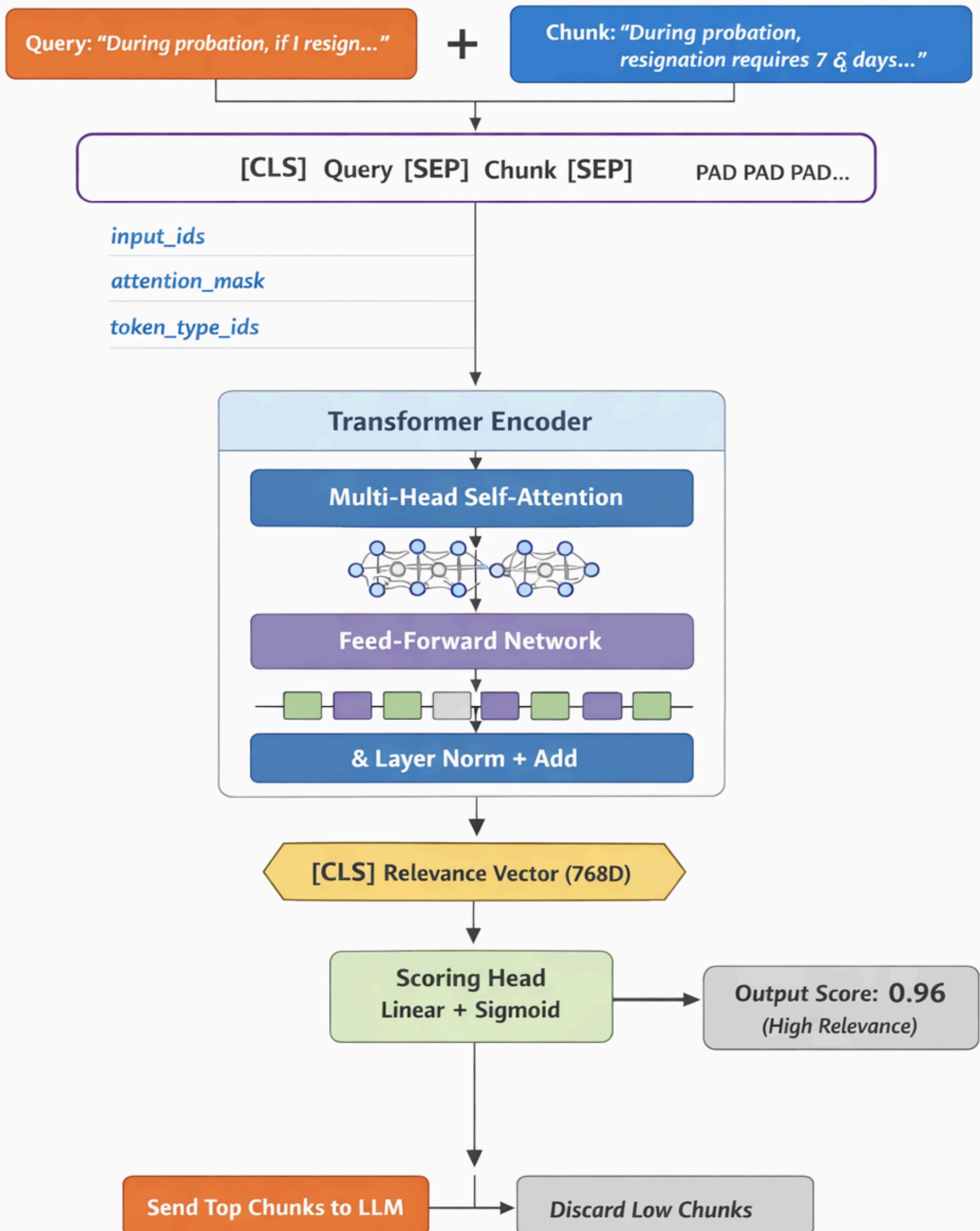
- uses the **Transformer encoder only**
- uses **multi-head self-attention**
- processes **query and chunk together as one sequence**
- outputs **a single scalar relevance score per chunk**

The word *cross* refers to **cross-token interaction** between query tokens and chunk tokens, not encoder-decoder attention.

Position of Re-Ranking in the RAG Pipeline

- User submits query
 - Query is embedded
 - Vector database retrieves Top-K chunks
 - Each chunk is evaluated by a **cross-encoder re-ranker**
 - Chunks are sorted by relevance score
 - Top-N chunks are sent to the LLM
-

Cross-Encoder Re-Ranking



Example Setup (HR Policy Use Case)

User Query

“During probation, if I resign, what notice period do I need to serve?”

Retrieved Chunks (Different Lengths)

- **Chunk 1 (short, correct)** “During probation, resignation requires 7 days notice.”
- **Chunk 2 (longer, partially related but wrong scope)** “After confirmation of employment, employees are required to serve a notice period of 30 days. This policy applies to confirmed staff and does not cover probationary exits.”

Vector search retrieves both because both mention *resignation* and *notice period*.

Joint Input Construction (Cross-Encoder Input)

For each chunk, the model builds a **single combined input sequence**:

```
[CLS] Query [SEP] Chunk [SEP]
```

This allows **full semantic interaction** between query tokens and chunk tokens.

Tokenization and Length Difference

Transformers operate on **subword tokens**.

- Query $\approx$ 18 tokens
- Chunk 1 $\approx$ 10 tokens $\rightarrow$ total $\approx$ 29 tokens
- Chunk 2 $\approx$ 28 tokens $\rightarrow$ total $\approx$ 47 tokens

Different chunk lengths naturally produce **different sequence lengths**.

Padding and attention_mask

Re-ranking is done in batches, so all sequences must have the same length.

Assume:

```
max_sequence_length = 50
```

Padding:

- Chunk 1 $\rightarrow$ 21 padding tokens
- Chunk 2 $\rightarrow$ 3 padding tokens

attention_mask:

- real tokens $\rightarrow$ 1
- padding tokens $\rightarrow$ 0

padding tokens exist numerically but are **fully ignored during attention**.

Embedding Computation (Numeric)

For each token position:

```
Embedding = TokenEmbedding + PositionEmbedding + SegmentEmbedding
```

Example (shortened):

```
TokenEmbedding    = [ 0.12, -0.44,  0.31 ]
PositionEmbedding = [ 0.03,  0.01, -0.12 ]
SegmentEmbedding  = [ 0.02, -0.01,  0.04 ]

FinalEmbedding     = [ 0.17, -0.44,  0.23 ]
```

Final embedding matrix shapes:

- Chunk 1 $\rightarrow$ [50, 768]
- Chunk 2 $\rightarrow$ [50, 768]

Transformer Encoder with Multi-Head Self-Attention

The embeddings pass through multiple encoder layers (e.g., 12).

Inside each layer:

- Query, Key, Value matrices are computed
- Multi-head self-attention aligns tokens
- `attention_mask` forces padding attention to zero

Conceptually:

- “probation” ↔ “probation” → strong signal (Chunk 1)
- “probation” ↔ “confirmation” → mismatch (Chunk 2)
- “notice period” ↔ “7 days” vs “30 days” evaluated in context

Longer chunks do **not** automatically score higher; attention focuses on meaning, not size.

Final Encoder Output

After the last encoder layer:

$$H = [h_{CLS}, h_1, h_2, \dots, h_{50}]$$

Each vector has dimension 768.

Example:

- Chunk 1 → `h_CLS ≈ [0.91, -0.38, 0.44, ...]`
- Chunk 2 → `h_CLS ≈ [0.18, -0.62, 0.09, ...]`

All semantic reasoning is now compressed into `h_CLS`.

Scoring Using `h_CLS` (Linear Projection)

The relevance score is computed as:

$$\text{logit} = W \cdot h_{CLS} + b$$

Simplified numeric example:

$$\begin{aligned} h_{CLS} &= [0.9, -0.4, 0.6, 0.1, 0.7] \\ W &= [1.2, -0.8, 0.5, 0.3, 1.0] \\ b &= 0.2 \end{aligned}$$

Chunk 1:

$$\text{logit} \approx 2.63$$

Chunk 2:

$$\text{logit} \approx -0.55$$

Sigmoid Normalization

$$\text{score} = 1 / (1 + e^{(-\text{logit})})$$

- Chunk 1 → ≈ 0.93
- Chunk 2 → ≈ 0.36

Scores represent **relative relevance**.

Re-Ranking Decision

The model outputs scores; ranking happens outside the model.

Final ordering:

$$\text{Chunk 1 } (0.93) > \text{Chunk 2 } (0.36)$$

Only Chunk 1 is passed to the LLM.

Implementation Code (Add to Notes)

Simple Cross-Encoder Re-Ranking (Recommended)

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

query = "During probation, if I resign, what notice period do I need to serve?"
chunks = [
    "During probation, resignation requires 7 days notice.",
    "After confirmation of employment, employees must serve a notice period of 30 days."
]

pairs = [(query, c) for c in chunks]
scores = reranker.predict(pairs)

ranked = sorted(zip(chunks, scores), key=lambda x: x[1], reverse=True)

for chunk, score in ranked:
    print(score, chunk)
```

Key Conceptual Takeaways

- Re-ranking focuses on **precision**, not recall
- Cross-encoders enable **full query–chunk interaction**
- Chunk length does **not** bias relevance
- Padding exists only for batching and is ignored
- Token vectors matter during attention, not after encoding
- `[CLS]` aggregates all reasoning
- Linear projection converts meaning into a score
- Sorting scores produces final re-ranking

5.10 Retrieval Strategies in Action – Pune City Example

Chunk 1 – Historical Spots: “Shaniwar Wada is a historic fortification in the city center. It was built in 1732 and is a popular tourist destination known for its cultural heritage.”

Chunk 2 – Museums: “Raja Dinkar Kelkar Museum displays a vast collection of everyday Indian artifacts and art pieces. It is located in Shukrawar Peth and attracts history enthusiasts.”

Chunk 3 – Nature Walks: “Vetal Tekdi is a beautiful hill in Pune where people enjoy early morning walks, bird watching, and sunset views.”

Chunk 4 – Religious & Wellness: “Osho Ashram in Koregaon Park is a peaceful retreat for meditation, yoga, and spiritual wellness. It attracts global visitors.”

Chunk 5 – Food & Cafés: “FC Road and JM Road are famous for street food, fast food joints, and young crowd cafés that remain lively into the evening.”

Chunk 6 – Adventure: “Sinhagad Fort is a popular trekking destination on the outskirts of Pune, known for its history, sunrise views, and local snacks at the top.”

Chunk 7 – Cultural Events: “Pune hosts the Ganesh Chaturthi festival on a grand scale with large processions, music, and dance that attract thousands every year.”

Chunk 8 – Educational Landmarks: “Fergusson College and Savitribai Phule Pune University are not just academic institutions, but also heritage campuses with beautiful architecture.”

 User Query:

“What are the best places to visit in Pune?”

Simulated Outputs from Each Retriever

1. Dense Retriever

- ◆ **Output:** Returns top 3 semantically relevant chunks:
 - Chunk 1: Shaniwar Wada (historic site)
 - Chunk 3: Vetal Tekdi (nature visit)
 - Chunk 6: Sinhagad Fort (trekking spot)
 - 💡 **Why:** These match the **meaning** of “places to visit” even if they don’t use that exact phrase. No strict keywords needed.
-

◆ **2. MMR Retriever**

- ◆ **Output (diverse selection):**
 - Chunk 1: Shaniwar Wada (historic)
 - Chunk 3: Vetal Tekdi (nature)
 - Chunk 4: Osho Ashram (spiritual)
 - Chunk 5: FC Road (food/café)
 - Chunk 6: Sinhagad Fort (adventure)

💡 **Why:** Returns a **variety** of places — not just forts or nature — ensuring different **types of experiences**.

◆ **3. Multi-Query Retriever**

- ◆ **Reformulated Queries:**
 - “Popular tourist spots in Pune”
 - “Fun things to do in Pune”
 - “Places locals love in Pune”
 - “Nature or spiritual retreats in Pune”
- ◆ **Output (merged):**
 - Chunk 1: Shaniwar Wada
 - Chunk 4: Osho Ashram
 - Chunk 5: FC Road
 - Chunk 3: Vetal Tekdi
 - Chunk 7: Ganesh Festival

💡 **Why:** Each reformulated query brings **different kinds** of places. Final result is **broadier** and reflects different intents.

◆ **4. Hybrid Retriever**

Assume query included: “...places like museums, hills, or heritage locations”

- ◆ **Output:**
 - Chunk 2: Museum (keyword: "museum")
 - Chunk 3: Vetal Tekdi (semantic: "hills")
 - Chunk 8: Fergusson College (heritage, keyword match)

💡 **Why:** Matches **keywords** (museum, heritage) + **semantic similarity** (hills = nature).

◆ **5. Self-Query Retriever**

Assume metadata per chunk:

Chunk	Category
1	Historical
2	Museum
3	Nature
4	Wellness
5	Food
6	Adventure
7	Festival
8	Educational

Query: “Show me only historical and nature places to visit in Pune”

- ◆ **Output:**

- Chunk 1: Shaniwar Wada
 - Chunk 3: Vetel Tekdi
 - Chunk 6: Sinhagad Fort
- 🗨️ **Why:** Applies metadata filters for **categories**, returning only matching chunks.

◆ **6. Contextual Compression Retriever**

- ◆ **Raw Chunks Retrieved:** Chunks 1–6 (based on relevance)
- ◆ **Output After Compression:**

“Visit Shaniwar Wada for history, Vetel Tekdi for nature, Sinhagad Fort for trekking, FC Road for food, and Osho Ashram for wellness.”

- 🗨️ **Why:** Instead of feeding full chunks, the retriever **extracts only relevant lines** and builds a **compact, token-efficient summary**.

■ **Summary Table**

Retriever Type	Example Output Summary	Key Benefit
Dense Retriever	Historical + Nature + Adventure	Semantically similar results
MMR Retriever	History + Nature + Food + Wellness + Trekking	Balanced, non-redundant selection
Multi-Query Retriever	Adds Cultural & Local favorites	Broader, multi-intent recall
Hybrid Retriever	Museum + Hill + Heritage Campus	Combines exact & semantic match
Self-Query Retriever	Filters by categories like historical, nature, etc.	Structured, user-controlled filtering
Contextual Compression	Returns only key phrases from all retrieved content	Token-efficient, focused information

✅ **Final Conclusion**

- ◆ With **8 chunks**, all retrievers have the chance to **show their strengths**:
- Dense → pure semantic match
- MMR → diverse activities
- Multi-Query → query expansion
- Hybrid → combines meaning + keywords
- Self-Query → filtered by user-driven categories
- Compression → compact result for generation

6. Query Transformation Techniques

Query transformation techniques improve retrieval by **modifying the user query before it is embedded** so that it accurately represents the user’s **true intent** in a retrieval-friendly form.

They are applied **before query embedding**, because embeddings are sensitive to ambiguity, missing context, and multi-intent phrasing. If the query itself is poor, even the best retriever will fail.

Why Query Transformation Is Important

In real RAG systems, user queries are often:

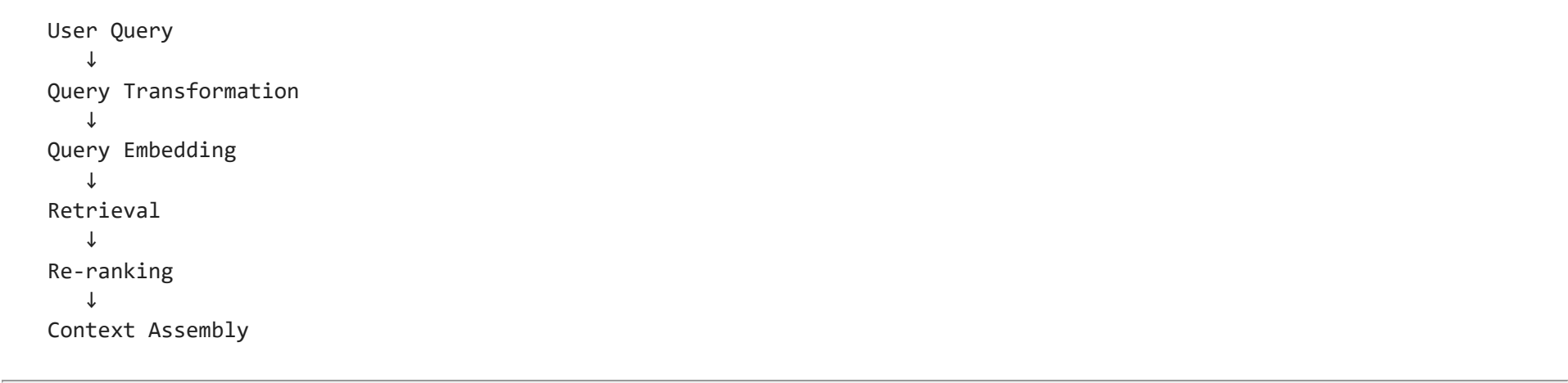
- short or vague
- conversational
- missing domain terms
- multi-part

Query transformation helps to:

- increase **recall** (find all relevant chunks)
- improve **precision** (avoid irrelevant chunks)
- reduce **semantic drift**
- improve **retrieval faithfulness**

Think of it as **preparing the question** so retrieval works reliably.

Where It Fits in the RAG Pipeline



6.1 Query Rewriting

Query rewriting reformulates a query into a **clear and explicit version** without changing its meaning.

It fixes vague wording, missing scope, and informal language so that the embedding captures the correct intent.

Example

- Original: "Tell me about SunLife leave"
- Rewritten: "What is the employee leave policy for SunLife?"

Benefits

- Better query embeddings
- Fewer irrelevant results
- Stronger grounding in policy documents

Risks

- Poor rewriting can slightly alter meaning
- Requires guardrails if using an LLM

When to Use

- Short or vague queries
- Conversational interfaces
- HR, support, policy systems

```
def rewrite_query(llm, query):  
    return llm(f"Rewrite clearly for document retrieval: {query}")
```

6.2 Query Expansion

Query expansion adds **synonyms and domain terms** to the query to improve recall when document wording differs from user wording.

Example

- Original: "leave policy"
- Expanded: "leave policy PTO time off vacation paid leave"

Benefits

- Improves recall@k
- Very effective with hybrid (dense + BM25) retrieval

Risks

- Over-expansion can add noise

When to Use

- Legal, HR, insurance domains
- Inconsistent document vocabulary
- Keyword-sensitive search

```
def expand_query(query):  
    return query + " PTO time off vacation"
```

6.3 Query Decomposition

Query decomposition splits a **multi-intent query** into multiple single-intent sub-queries so that each intent retrieves focused chunks.

Example Original: "What is the leave policy and how many days can be carried forward?"

Decomposed:

1. "What is the leave policy?"
2. "What are the carry-forward rules?"

Benefits

- Better coverage for complex questions
- Prevents one intent from hiding others

Risks

- Multiple retrieval calls
- Requires merging and deduplication

When to Use

- Policy and compliance questions
- Long or compound queries

```
def decompose_query(llm, query):  
    return llm(f"Split into minimal sub-questions: {query}")
```

6.4 How These Techniques Are Used Together

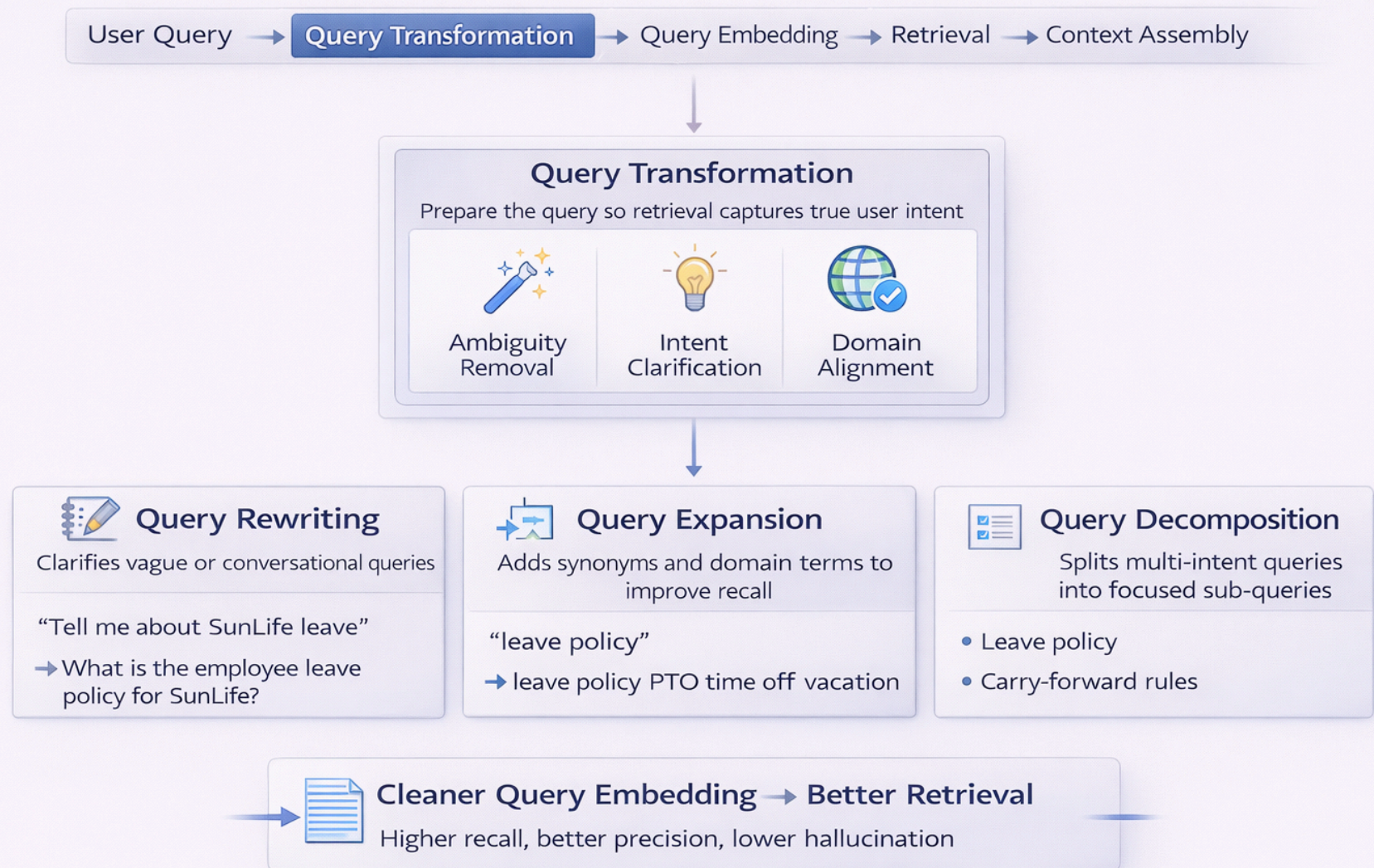
In practice, systems rarely use just one technique.

A common and effective pattern:

1. Rewrite to remove ambiguity
2. Decompose if multiple intents exist
3. Expand lightly for domain terms
4. Embed and retrieve
5. Merge and deduplicate results

This balances **high recall**, **low redundancy**, and **strong grounding**.

Query Transformation Techniques (Before Retrieval)



7. Prompt Engineering in RAG

Prompt engineering in RAG is the process of **structuring instructions, retrieved context, and user questions** so that the LLM produces **grounded, accurate, and predictable responses**.

Even with perfect retrieval, weak prompt design can cause:

- hallucinations
- ignored context
- inconsistent formatting
- use of external knowledge

In RAG systems, prompt engineering is the **final control layer before generation**.

7.1 Why Prompt Engineering Is Critical in RAG

LLMs do not inherently know:

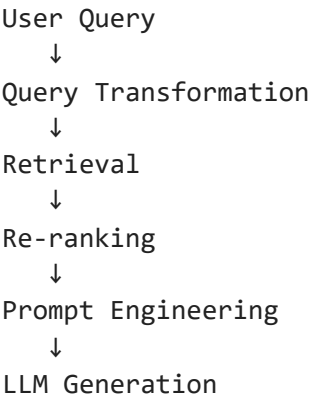
- which retrieved chunk is authoritative
- whether missing information should be guessed
- which tone or format is required

Prompt engineering ensures:

- answers come **only from retrieved data**
- fallback behavior is safe and predictable
- output is structured for UI or APIs

Retrieval controls *what the model sees* Prompt engineering controls *how the model uses it*

7.2 Where Prompt Engineering Fits in the RAG Pipeline



7.3 Prompt Roles in Chat-Based RAG

Modern LLM APIs use structured message roles.

Role	Purpose
system	Global behavior, rules, constraints
user	Retrieved context + question
assistant	Previous responses (multi-turn only)

7.4 System Prompt (RAG Context)

The system prompt defines the **global identity, behavior, and constraints** of the language model for the entire conversation. It acts as a **policy layer** that governs *how the model thinks and responds*, independent of the user’s question or retrieved data.

In simple terms, the system prompt controls:

- the role of the model
- its responsibilities
- allowed and forbidden behaviors
- how to handle missing or insufficient information

The system prompt is **not data** and **not contextual content**. It is **behavioral control logic**.

Priority and Role in Prompt Hierarchy

The system prompt has the **highest priority** in the prompt hierarchy.

- it cannot be overridden by user instructions
- all other prompts are interpreted under its rules
- retrieved content is treated as evidence, not authority

This makes the system prompt the **final guardrail** in a RAG pipeline.

Why System Prompt Is Essential in RAG

In a RAG system, the LLM has access to two knowledge sources:

- pretrained model knowledge
- retrieved document chunks

Without a strict system prompt, the model may:

- answer using pretrained knowledge instead of retrieved context
- guess or infer missing facts
- combine retrieved information with external knowledge
- produce confident but incorrect responses

The system prompt prevents this by **explicitly restricting the knowledge scope** and enforcing **grounded, evidence-based reasoning**.

Example: System Prompt (HR RAG)

You are an HR policy assistant.

Use only the information provided in the context.
Do not use external knowledge.
Do not guess or assume missing details.

If the answer is not found in the context, respond exactly with:
"Not found in the context."

Keep responses factual and concise.

Why This System Prompt Works

- clearly defines the model's role
 - enforces strict grounding to retrieved context
 - specifies fallback behavior for missing information
 - prevents hallucination and assumption
 - shifts the model from creative generation to controlled reasoning
-

7.5 User Prompt (RAG Context)

The user prompt provides the **task-specific input** to the language model. In a RAG system, it supplies **what the model should work on**, not **how it should behave**.

The user prompt typically contains:

- retrieved document chunks (context)
- the user's question

It answers:

- *what information is available*
- *what question needs to be answered*

The user prompt is **data-driven**, not behavioral.

Role of User Prompt in Prompt Hierarchy

The user prompt operates **under the rules defined by the system prompt**.

- it cannot override system-level constraints
- it does not define safety, role, or behavior
- it is interpreted strictly within system prompt boundaries

In RAG, the user prompt supplies **evidence**, not authority.

Why User Prompt Is Important in RAG

The quality of the user prompt directly affects:

- grounding accuracy
- relevance of answers
- hallucination control

A well-structured user prompt ensures that:

- the model reasons only over retrieved content
- answers are traceable to source chunks
- responses stay aligned with the question intent

Poorly structured user prompts often lead to:

- mixed or incomplete answers
 - over-generalization
 - incorrect assumptions
-

Standard User Prompt Structure (RAG)

Context:

1. Retrieved chunk 1
2. Retrieved chunk 2

Question:

User question

This structure explicitly separates:

- **evidence** (context)
 - **task** (question)
-

Example: User Prompt (HR RAG)

Context:

1. Employees are eligible for leave after completing 90 days of service.
2. Leave carry-forward is limited to 15 days per year.

Question:

When can a new employee apply for leave?

Why This User Prompt Works

- context is clearly defined and enumerated
 - question is unambiguous
 - no behavioral instructions are included
 - model is forced to reason strictly from provided data
-

What Should NOT Go in User Prompt

The user prompt should **not include**:

- role definitions
- behavioral or safety rules
- fallback or refusal logic
- instructions like “do not hallucinate”

These belong exclusively in the **system prompt**.

7.6 Standard Prompt Construction Pattern

System:

[Behavior and constraints]

Context:

[Retrieved chunks]

Question:

[User query]

Instruction:

[Fallback / format rules]

This separation:

- enforces grounding
 - reduces hallucination
 - improves precision
-

7.7 Instruction Prompting

Instruction prompting explicitly tells the model **what to do and what to avoid**.

Example Instruction

Only answer using the provided context.

Do not use external knowledge.

If the answer is not found, say: "Not found in the context."

Simple Code Snippet

```
system_prompt = (  
    "Answer only using the provided context. "  
    "Do not use external knowledge. "  
    "If the answer is not found, say: 'Not found in the context.'"  
)
```

7.8 Role Prompting

Role prompting assigns a **domain identity** to the model.

Example

You are a legal assistant trained to interpret contracts.
Use only the provided context.
Do not make assumptions.

Why It Works

- narrows reasoning scope
 - aligns vocabulary and tone
 - reduces creative output
-

7.9 Output Formatting Control

Used when responses must be **machine-readable or UI-ready**.

JSON Output Example

```
Instruction:
Respond in the following JSON format:
{
  "plan_name": "",
  "eligibility": "",
  "vesting_period": ""
}
output_format_instruction = (
  "Return the answer strictly in JSON format with keys: "
  "plan_name, eligibility, vesting_period."
)
```

Bullet / Table Output Example

Return the answer in bullet points.
Respond in a table with columns: Benefit | Description

7.10 Zero-shot and Few-shot Prompting

These techniques control **style and structure**, not retrieval.

7.10.1 Zero-shot Prompting

Task is described **without examples**.

Example (Zero-shot)

System Prompt

You are an HR assistant.
Answer only using the context.
If the answer is not found, say: "Not found in the context."

User Prompt

Context:
1. Employees are eligible for leave after 90 days.

Question:
When can employees apply for leave?

Expected Output

Employees can apply for leave after completing 90 days of service.

When to Use

- simple factual questions
 - obvious output format
 - low latency needs
-

7.10.2 Few-shot Prompting

Includes **example Q&A pairs** to guide format or tone.

Example (Few-shot)

Context:
1. Employees are eligible for leave after 90 days.
2. Carry-forward limit is 15 days.

Example:
Q: When are employees eligible for leave?
A:
- Employees are eligible after 90 days.

Question:
What is the carry-forward limit?

Expected Output

- The carry-forward limit is 15 days.

When to Use

- strict formatting (bullets, JSON, tables)
- compliance systems
- reusable templates

Zero-shot vs Few-shot (Quick View)

Aspect	Zero-shot	Few-shot
Examples	No	Yes
Tokens	Low	Higher
Consistency	Medium	High
Best for	Simple Q&A	Structured output

7.11 Context Window and Token Budget Management

RAG answers must fit inside the model’s **context window**:

- system prompt
- retrieved context (Top-K chunks)
- chat history (if any)
- user question
- tool outputs (optional)
- generated answer

If token budget is not managed, prompts may get truncated, causing:

- missing critical context
- incomplete answers
- unpredictable behavior

Recommended Practices

- Keep chunk size moderate (often 100–300 tokens) so you can include multiple chunks
- Reserve output budget explicitly (do not spend all tokens on context)
- Prefer fewer high-quality chunks over many noisy chunks
- Use summarization or compression if context is too long

Simple Code Idea (Token Budgeting Concept)

MAX_TOTAL_TOKENS = 8000
RESERVED_FOR_OUTPUT = 600
MAX_INPUT_TOKENS = MAX_TOTAL_TOKENS - RESERVED_FOR_OUTPUT

7.12 Generation Parameters for RAG (Temperature, Top-p, Max Tokens, Stop)

Generation parameters control **how the LLM writes the answer**. They should be documented inside Prompt Engineering because they directly affect:

- determinism
- hallucination rate

- verbosity
- repeatability

Common Parameters

- `temperature` : randomness (RAG usually low)
- `top_p` : nucleus sampling (often left at 1.0)
- `max_tokens` (or `max_output_tokens`): controls output length
- `stop` : forces stopping at a token/sequence
- `frequency_penalty` : reduces repetition (small values only)
- `presence_penalty` : encourages novelty (usually keep 0.0 for factual RAG)

Recommended Defaults (Typical RAG)

- `temperature = 0.0 to 0.3`
- `top_p = 1.0`
- `presence_penalty = 0.0`
- `frequency_penalty = 0.0 to 0.5`
- output length limited using `max_tokens`

Simple Code Snippet

```
gen_params = {  
    "temperature": 0.2,  
    "top_p": 1.0,  
    "max_tokens": 400,  
    "presence_penalty": 0.0,  
    "frequency_penalty": 0.2,  
    # "stop": ["\n\n###"] # optional  
}
```

Temperature

Controls how **random or deterministic** the model's output is by influencing how the next token is chosen.

- **Low temperature** → model consistently selects the most probable next token
- **High temperature** → model samples from less probable tokens, increasing variation

Why this matters in RAG

RAG systems are designed to answer **from retrieved facts**, not to invent or creatively paraphrase. Higher temperature increases the chance that the model:

- rephrases content inaccurately
- introduces unsupported details
- blends external or pretrained knowledge

Recommended usage in RAG

- `0.0` → strict, fully deterministic (best for compliance, legal, HR, finance)
- `0.1–0.3` → slight flexibility while staying factual (good for explanations)
- Avoid values above `0.3` in factual RAG pipelines

Key intuition

Temperature controls *how safely the model speaks*, not *what it knows*.

top_p (Nucleus Sampling)

Limits token selection to the most probable tokens.

- Usually left at `1.0` in RAG
- Rarely tuned unless fine control is needed

max_tokens / max_output_tokens

Sets the maximum length of the generated answer.

- Prevents overly long responses
 - Controls cost and verbosity
 - Output tokens must fit within the model's context window
-

7.13 Chain-of-Thought Prompting

Chain-of-thought prompting enables the model to perform **multi-step internal reasoning** before producing a final answer. It is particularly useful for queries that require **logical sequencing, eligibility evaluation, or procedural guidance**.

In **production RAG systems**, chain-of-thought is used **internally to improve reasoning quality**, but **raw reasoning steps are not exposed** to the end user. Instead, the model is instructed to return **structured or summarized outputs**.

When Chain-of-Thought Is Useful

- procedural or how-to questions
- eligibility and rule-based decisions
- multi-condition reasoning
- step-wise problem solving

Example

Context:
Sinhagad Fort is 30 km from Pune Station.
Buses run from Swargate.

Question:
How can I reach Sinhagad Fort from Pune Station?

Instruction:
Provide the steps as a numbered list.
Do not add assumptions or external information.

7.14 Hallucination Reduction Techniques

Technique	Purpose
Explicit grounding	Prevent external knowledge
Fallback response	Safe behavior when answer missing
Low temperature (0–0.3)	Reduce randomness
Context delimiters	Avoid mixing chunks
Chunk limits	Prevent dilution
Role prompting	Domain control

7.15 Multi-turn Prompting in RAG

Conversational RAG combines:

- chat history
- newly retrieved context
- latest user query

This helps resolve references like: “this”, “that”, “which one”.

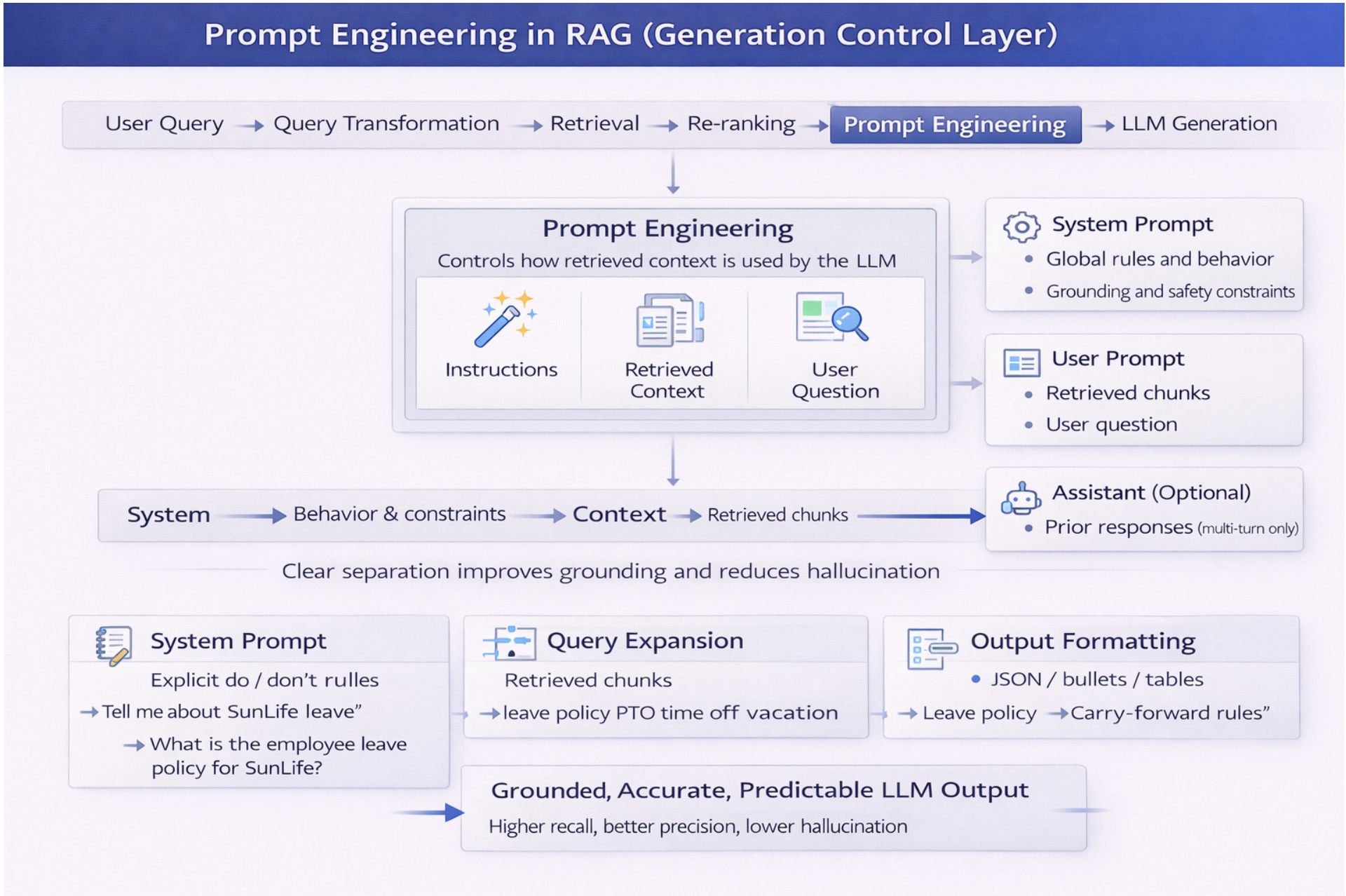
Simple Code Idea

```
messages = history + [  
    {"role": "system", "content": system_prompt},  
    {"role": "user", "content": context_block + "\n\nQuestion:\n" + question}  
]
```

7.16 Summary: Prompt Engineering in RAG

Technique	Purpose
System prompt	Global behavior control
User prompt	Supplies context + question
Instruction prompting	Enforces rules
Role prompting	Domain alignment

Technique	Purpose
Output formatting	UI/API readiness
Zero/Few-shot	Style consistency
Token budgeting	Prevent truncation, control cost
Generation parameters	Control randomness and output length
Multi-turn prompting	Conversational flow



9. Tools and Frameworks for RAG

A production-ready Retrieval-Augmented Generation (RAG) system relies on a **combination of tools** across multiple stages of the pipeline, including preprocessing, embedding, retrieval, orchestration, evaluation, and observability.

These tools can be chosen from:

- open-source ecosystems
- Azure-native services
- AWS-native services

The choice depends on scalability, compliance, cost, and integration requirements.

9.1 Preprocessing and Chunking

These tools are used to **extract, clean, normalize, and chunk documents** before embedding.

Common Tools

- **spaCy** – sentence segmentation, tokenization, NLP preprocessing
- **NLTK** – basic NLP utilities (sentence and word tokenizers)
- **PyMuPDF** – PDF text extraction

- **Apache Tika** – multi-format document parsing
- **Amazon Texttract** – OCR and structured data extraction from scanned documents
- **Azure Form Recognizer (Document Intelligence)** – OCR, tables, forms, layout-aware extraction

Used for: PDFs, Word files, scanned documents, HTML, contracts, and reports.

9.2 Embedding Models

Embedding models convert text chunks into dense vector representations used for similarity search.

Common Options

- **OpenAI Embeddings** (`text-embedding-ada-002`)
- **Azure OpenAI Embeddings**
- **Cohere Embeddings**
- **HuggingFace Transformers** (MiniLM, BGE, E5, MPNet, etc.)
- **Amazon Titan Embeddings**
- **SageMaker JumpStart embedding models**

Selection depends on:

- dimensionality
 - latency
 - multilingual support
 - cost
 - on-prem vs managed deployment
-

9.3 Vector Databases and Search Engines

Vector databases store embeddings and enable **fast similarity search**, often combined with metadata filtering or hybrid search.

Common Vector Stores

- **FAISS** – local, in-memory ANN search (research and PoC)
- **Pinecone** – managed, cloud-native vector database
- **Weaviate** – open-source and managed, hybrid search support
- **Chroma** – lightweight local vector store
- **Milvus** – distributed, high-scale vector database
- **Qdrant** – open-source, production-ready vector DB
- **Redis (Vector Similarity)** – low-latency vector search with metadata

Cloud-Native Search Services

- **Azure AI Search** – vector + keyword + semantic hybrid search
 - **Amazon OpenSearch Service (k-NN)** – vector + keyword search
 - **Amazon Kendra** – enterprise semantic + keyword search with connectors
-

9.4 RAG Orchestration Frameworks

Orchestration frameworks manage the **end-to-end RAG workflow**, including retrieval, prompt construction, and chaining logic.

Popular Frameworks

- **LangChain** – flexible chaining, retrievers, agents, tools
- **LlamaIndex** – data ingestion, indexing, query engines
- **Haystack** – search-first RAG pipelines
- **Semantic Kernel** – Microsoft’s orchestration framework for LLM apps
- **Agents for Amazon Bedrock** – AWS-native agentic orchestration

Used for:

- chaining retrieval + generation
 - tool calling
 - agent-based RAG workflows
-

9.5 Evaluation, Monitoring, and Observability

These tools help evaluate RAG quality and monitor production behavior.

Evaluation Tools

- **RAGAS** – retrieval and generation quality metrics
- **TruLens** – feedback-driven evaluation for RAG pipelines
- **LangChain Evaluation Toolkit**
- **OpenAI Evals**

Monitoring and Observability

- **PromptLayer** – prompt versioning and monitoring
- **Azure Monitor + Application Insights** – logs, latency, failures
- **Amazon CloudWatch** – metrics, logs, alarms

Used for:

- hallucination detection
- performance regression tracking
- cost and latency monitoring
- compliance auditing

9.6 Tool Selection Guidelines

When selecting tools for a RAG system, consider:

- **Deployment environment** (Azure, AWS, on-prem, hybrid)
- **Data sensitivity and compliance**
- **Latency and throughput requirements**
- **Cost constraints**
- **Team familiarity and ecosystem integration**

RAG systems are **modular by design**, and tools can be mixed across ecosystems if needed.

10. Security, Privacy, and Compliance

In enterprise-grade RAG systems, ensuring the **security, privacy, and compliance** of both input documents and generated outputs is critical—especially in domains like **finance, healthcare, HR, and legal**. Without proper controls, systems risk:

- Leaking **sensitive information (PII)**
- Violating **regulatory standards (e.g., GDPR, HIPAA)**
- Producing **untraceable or unauthorized outputs**

◆ 10.1 Key Risks in RAG

Risk Type	Description
PII Leakage	Retrieval or generation may surface names, emails, salaries, etc.
Unauthorized Access	Users may access content beyond their role or business unit
Prompt Injection	Malicious inputs may override system behavior
Unlogged Generation	Generated answers may lack traceability for audits
Data Residency Violation	Embeddings or logs may be stored outside permitted regions

◆ 10.2 PII Handling in RAG

 What is PII?

Personally Identifiable Information (PII) includes names, contact info, IDs, financials, or anything that can uniquely identify a person.

PII Type	Examples
Identifiers	Full name, Employee ID, SSN, PAN
Contact Info	Email, phone, address
Financial	Bank details, salary, investment records
Health Info	Insurance ID, prescriptions, medical conditions

⚠️ Example of Unsafe Output:

“John Smith (Employee ID: 4489) must submit his health form to HR at john.smith@abc.com.”

✖ Violates privacy—exposes identity, email, and health-related context.

✅ How to Avoid PII Leakage

Stage	Strategy
Preprocessing	Redact or mask PII before chunking (use regex, spaCy, or Presidio)
Chunk Tagging	Label chunks with <code>"sensitivity": "high"</code> and restrict their retrieval
Prompt Guardrails	Instruct the model not to reveal names, emails, IDs
Post-filtering	Scan outputs using moderation tools (OpenAI Moderation, Azure Guardrails)

Prompt Example:

Do not output any names, email addresses, phone numbers, or personal identifiers.

◆ 10.3 Access Control and Authorization

Implement **role-based filtering at retrieval time** using chunk metadata such as:

```
{
  "department": "HR",
  "role": "internal",
  "client": "ABC Corp"
}
```

- Enforce filters using tools like:
 - LangChain Retriever Filters
 - Azure AI Search Filters
 - Weaviate / Pinecone metadata filtering

Authentication via:

- Azure Active Directory (AAD)
- AWS Cognito
- OAuth2 / SAML / Okta

◆ 10.4 Encryption and Logging

Aspect	Recommendation
Data at Rest	Use encrypted vector stores (e.g., Azure AI Search, Pinecone)
In Transit	TLS 1.2+ for all API communications
Key Management	Use Azure Key Vault or AWS KMS for key rotation
Audit Logging	Log user ID, timestamp, retrieved chunks, and final output
Retention	Store logs for at least 90 days for audit purposes

◆ 10.5 Prompt & Output Guardrails

Prevent unsafe model behavior with **instructional prompting** and post-checks:

✅ Prompt Guardrails

Only answer from the provided context. Do not guess. Never return personal names or contact details.

✅ Output Moderation Tools

Tool/Service	Use Case
OpenAI Moderation API	Flag unsafe or policy-violating output
Azure Content Filter	Enterprise-level content control
AWS Bedrock Guardrails	Configure safety rules and filters

◆ 10.6 Regulatory Compliance Mapping

Framework	Applies To	What to Ensure
GDPR	EU data privacy	No unauthorized storage; user right to erasure
HIPAA	U.S. healthcare data	Mask PHI; log access; secure storage
SOC 2	SaaS & data processors	Logging, access control, change tracking
ISO 27001	Information security programs	Enterprise risk and compliance audits
FINRA / SEC	U.S. financial services	Preserve audit trails for generated responses

◆ 10.7 Summary: Enterprise RAG Security Checklist

Area	Control
Access Filtering	Use metadata (e.g., department, region) for retrieval scoping
PII Protection	Preprocess, redact, and guard prompt/output
Authentication	AAD / Cognito / OAuth2 integration
Audit Logging	Log context + answer per user/session
Prompt Hardening	Add "no guessing" / "no external knowledge" instructions
Data Residency	Use region-specific models, storage, and logging

11. Best Practices for Building RAG Systems

Designing and deploying Retrieval-Augmented Generation (RAG) systems in production requires balancing **accuracy**, **efficiency**, **security**, and **scalability**. The following best practices cover each critical stage of the RAG pipeline.

11.1 Chunking and Ingestion

- Use **overlapping sliding windows** (e.g., 100–300 tokens with 10–20% overlap) to preserve semantic continuity
- Apply **semantic chunking** (headings, sections, paragraphs) instead of fixed-length splits when document structure matters
- Tag chunks with **metadata** such as `source`, `doc_type`, `access_level`, `client_id`, and `version` to enable filtering and access control
- Redact or mask **PII and sensitive data** before embedding (especially for HR, healthcare, and legal documents)

11.2 Embedding and Vector Storage

- Choose **domain-optimized embedding models** (e.g., OpenAI embedding models, BGE, Cohere)
- Normalize vectors (L2 normalization) when using cosine similarity
- Use a **production-grade vector database** such as **Pinecone**, **Weaviate**, **Milvus**, or **Azure AI Search**
- Index embeddings with **metadata filters** to support role-based and scoped retrieval

11.3 Retrieval Strategies

- Prefer **MMR** or **Hybrid Retrieval** (dense + keyword) over pure Top-K to improve diversity and precision
- Implement **Self-Query Retrieval** when metadata is rich and consistent
- Tune `K` dynamically (e.g., Top-3 vs Top-10) based on query type and available token budget
- Track **recall@K** and regularly evaluate retriever coverage

11.4 Prompt Engineering

- Use clear **system-level instructions**, for example: *"Only answer using the context below. If unsure, say 'Not found in the context.'"*
- Format context with **delimiters or numbering** to clearly separate chunks

- Apply **instruction prompting** and **output formatting** (JSON, bullet points, tables)
 - Use **role prompting** to anchor domain-specific behavior (HR assistant, legal analyst, etc.)
 - For multi-step queries, enable **controlled reasoning** without exposing internal chain-of-thought
-

11.5 Evaluation and Monitoring

- Evaluate **retriever** and **generator** components independently
 - Use tools such as **RAGAS**, **TruLens**, and **OpenAI Evals** to measure faithfulness, relevance, and coverage
 - Include **no-answer test cases** to validate fallback behavior
 - Log **retrieved chunks**, **final answers**, and **user input** for debugging, audits, and traceability
-

11.6 Performance and Scaling

- Keep context length within the model's token limits
 - Apply **context compression** when needed (extractive summarization, sentence selection)
 - Use **asynchronous retrieval and caching** to reduce latency for repeated queries
 - Monitor token usage and optimize prompt size to balance cost and response time
-

11.7 Security and Privacy

- Enforce **metadata-based access control** at retrieval time
 - Apply **prompt sanitization** to defend against prompt injection and jailbreaks
 - Store only **anonymized or masked logs**
 - Use **region-specific models** to meet GDPR, HIPAA, and ISO requirements
 - Add strict fallback rules such as: *"If the answer is not found in the context, say 'Not found.' Do not guess."*
-

11.8 Maintenance and Continuous Improvement

- Refresh embeddings whenever source documents change
 - Version control prompt templates, retrievers, and configurations
 - Continuously tune retrieval strategies using QA scores and user feedback
 - Implement **feedback loops** so users can rate or flag answers for ongoing improvement
-

Final Thought

A high-quality RAG system is not just about retrieving relevant context—it is about delivering **safe, interpretable, compliant, and cost-effective responses** at scale. Following these best practices helps ensure a smooth transition from prototype to reliable production deployment.

Best Practices for Building RAG Systems

Accuracy • Scalability • Security • Reliability

Chunking & Ingestion



- Semantic + overlapping chunking
- Rich metadata tagging
- PII masking before embedding

Embedding Models



- Domain-optimized embeddings
- Consistent embedding space

Vector Databases



- Metadata filtering
- Scalable similarity search

Retrieval Strategies



- MMR for diversity
- Hybrid (dense + sparse)
- Dynamic Top-K

Prompt Engineering



- Strong system prompts
- Context delimiters
- Structured output formats

Evaluation & Monitoring



- Recall@K, faithfulness
- No-answer testing
- Audit logging

Performance & Scaling



- Context compression
- Caching & async retrieval
- Token & latency control

Security & Governance



- Metadata-based access control
- Prompt-injection defense
- GDPR / HIPAA compliance

Reliable RAG systems balance retrieval quality, safety, cost, and scale.

12 Real-World RAG Use Cases

1. HR Policy Assistant

Use Case: Employees ask questions like “How many casual leaves can I take?” or “What is the maternity policy?” **Solution:** RAG retrieves the latest HR policies and returns grounded, auditable answers.

✔ Domain: Human Resources ✔ Benefit: Avoids policy misinterpretation and ensures compliance

2. Customer Service Chatbot for Insurance

Use Case: Users ask, “What are the coverage limits under Plan A?” or “How do I withdraw from my 401(k)?” **Solution:** RAG pulls plan-specific details from PDF/SPD documents and presents personalized responses.

✔ Domain: Insurance / Financial Services ✔ Benefit: Reduces manual support load and improves accuracy

3. Legal Document Q&A Assistant

Use Case: Lawyers query, “Does this contract include a force majeure clause?” **Solution:** RAG searches across contracts, agreements, and case laws to surface exact clause-level content.

✔ Domain: Legal Tech ✔ Benefit: Faster due diligence, traceable legal support

4. Clinical Decision Support System

Use Case: Doctors ask, “What’s the recommended dosage for Drug X in patients over 60?” **Solution:** RAG pulls answers from FDA docs, internal hospital SOPs, and research papers.

✔ Domain: Healthcare / Life Sciences ✔ Benefit: Evidence-based, explainable recommendations for clinicians

5. University Knowledge Assistant

Use Case: Students ask, “What are the graduation requirements for Computer Science?” **Solution:** RAG searches student handbooks, curriculum guides, and FAQ PDFs.

✔ Domain: Education / Campus Operations ✔ Benefit: Unified access to decentralized academic knowledge

6. 💰 Internal Finance Policy Support

Use Case: Employees ask, “What’s the travel reimbursement limit for international trips?” **Solution:** RAG retrieves finance manuals and recent policy memos for accurate, contextual responses.

✔ Domain: Enterprise Finance Ops ✔ Benefit: Reduces support tickets, ensures policy adherence

7. 📄 Research Summarization Assistant

Use Case: Analysts ask, “What are the key findings across the latest 5 papers on generative AI?” **Solution:** RAG retrieves full-text research articles and generates a summary grounded in their content.

✔ Domain: R&D, Think Tanks, Academia ✔ Benefit: Speeds up knowledge extraction from lengthy documents

8. 🛡️ Compliance Audit Assistant

Use Case: Auditors ask, “Show me all mentions of GDPR compliance across client onboarding documents.” **Solution:** RAG scans all relevant onboarding SOPs, logs, and agreements to extract specific mentions.

✔ Domain: Compliance / Risk Management ✔ Benefit: Improves traceability and audit readiness

9. 🛒 E-commerce Product Support Bot

Use Case: Users ask, “Is this product returnable after 30 days?” **Solution:** RAG checks return policy PDFs, vendor SLAs, and product-specific terms.

✔ Domain: Retail / E-commerce ✔ Benefit: Self-service for customers, fewer escalations

10. 🏢 Enterprise Software Documentation Assistant

Use Case: Engineers ask, “How do I configure SSO in our internal ERP?” **Solution:** RAG retrieves internal docs, API guides, and SOPs to walk them through the process.

✔ Domain: IT / DevOps / SaaS ✔ Benefit: Reduces onboarding time and dependency on SMEs
